
SYSTEM DESIGN INTERVIEW SERIES · VOLUME I

The System Design Interview Playbook

A growing compendium of real interview problems, solved end-to-end: rigorous definitions, back-of-the-envelope mathematics, protocol design, and production-grade Rust implementations.

IN THIS VOLUME

- Chapter 01 — Designing a Real-Time Collaborative Text Editor (Google Docs / Notion)
- Chapter 02 — Designing a Maps & Navigation Service (Google Maps)
- Chapter 03 — Designing a Distributed Rate Limiter

Providence Salumu

First Edition · September 2026 · A living document — chapters are added incrementally

The System Design Interview Playbook

Volume I · First Edition

Written and typeset by **Providence Salumu**

Composed in Typst · Body set in Noto Serif · Display set in Noto Sans · Code set in Noto Sans Mono

REVISION HISTORY

v0.1	Chapter 1 — Real-Time Collaborative Text Editor	2026-09-05
v0.2	Chapter 2 — Maps & Navigation Service (Google Maps)	2026-09-05
v0.3	Chapter 3 — Distributed Rate Limiter	2026-09-05

© 2026 Providence Salumu. This document grows chapter by chapter;
each chapter is a self-contained walkthrough of one interview problem.

Contents

Preface	4
Conventions used in this book	4
How this book grows	5
1 Designing a Real-Time Collaborative Text Editor	6
1.1 The Problem Statement	6
1.2 Scope & Clarifying Questions	6
1.3 Functional Requirements	7
1.4 Non-Functional Requirements	8
1.5 Back-of-the-Envelope Estimation	9
1.6 The Core Challenge: Concurrent Editing	10
1.7 Version Vectors: Tracking What Each Replica Has Seen	13
1.8 API & Protocol Design	14
1.9 Data Model & Storage	16
1.10 High-Level Architecture	17
1.11 Deep Dive: The Edit Pipeline, Step by Step	18
1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson	19
1.13 Deep Dive: Rust Reference Implementation	20
1.14 Deep Dive: Presence & Live Cursors	23
1.15 Scaling & Sharding	23
1.16 Failure Modes & Recovery	24
1.17 Trade-offs & Alternatives	24
1.18 Observability & SLOs	25
1.19 Interview Wrap-Up	25
1.20 Summary & Further Reading	26
1.21 Chapter 1 Glossary	26
2 Designing a Maps & Navigation Service	28
2.1 The Problem Statement	28
2.2 Scope & Clarifying Questions	28
2.3 Functional Requirements	29
2.4 Non-Functional Requirements	30
2.5 Back-of-the-Envelope Estimation	30
2.6 Core Challenge I: Serving the Map	31
2.7 Core Challenge II: Modeling the Road Network	33
2.8 Core Challenge III: Shortest Paths at Planetary Scale	34
2.9 API & Protocol Design	36
2.10 Data Model & Storage	37
2.11 High-Level Architecture	38

2.12 Deep Dive: The Live Traffic Loop	40
2.13 Deep Dive: The Turn-by-Turn Navigation Session	41
2.14 Deep Dive: Rust Reference Implementations	42
2.15 Scaling & Geographic Sharding	48
2.16 Failure Modes & Recovery	49
2.17 Trade-offs & Alternatives	50
2.18 Observability & SLOs	50
2.19 Interview Wrap-Up	51
2.20 Summary & Further Reading	51
2.21 Chapter 2 Glossary	52
3 Designing a Distributed Rate Limiter	54
3.1 The Problem Statement	54
3.2 Scope & Clarifying Questions	55
3.3 Functional Requirements	55
3.4 Non-Functional Requirements	56
3.5 Back-of-the-Envelope Estimation	56
3.6 The Core Challenge: Counting Correctly Under Concurrency	57
3.7 The Algorithm Zoo	58
3.8 API & Protocol Design	59
3.9 Data Model & Storage	60
3.10 High-Level Architecture	61
3.11 Deep Dive: The Token Bucket, Precisely	61
3.12 Deep Dive: Distributed Enforcement & the Race	62
3.13 Deep Dive: Rust Reference Implementations	63
3.14 Scaling & Sharding	68
3.15 Failure Modes & Recovery	68
3.16 Trade-offs & Alternatives	69
3.17 Observability & SLOs	70
3.18 Interview Wrap-Up	70
3.19 Summary & Further Reading	71
3.20 Chapter 3 Glossary	71

Preface

This book exists because system design interviews reward a very specific skill that ordinary engineering work rarely exercises in full: *taking an ambiguous, enormous problem and shrinking it, in real time and out loud, into a coherent, defensible architecture*. Knowing how Netflix or Google Docs works is not enough; you must be able to **derive** such a design from first principles while an interviewer watches you think.

DEFINITION — System design interview

An interview format, common for senior engineering roles, in which the candidate is asked to design a large-scale software system (for example, “design YouTube” or “design a URL shortener”) within 45–60 minutes. There is no single correct answer. The interviewer evaluates how you clarify ambiguity, structure the problem, justify trade-offs, and reason about scale, failure, and consistency.

Each chapter of this book is a complete, self-contained walkthrough of one such problem. Every chapter follows the same battle-tested structure, which mirrors the natural arc of a strong interview performance:

THE CHAPTER FRAMEWORK

1. **Problem statement** — the prompt exactly as an interviewer would pose it.
2. **Clarifying questions & scope** — shrinking an ambiguous prompt into a concrete target.
3. **Functional requirements** — what the system must *do*.
4. **Non-functional requirements** — how *well* it must do it (scale, latency, availability).
5. **Back-of-the-envelope estimation** — arithmetic that sizes the problem before designing.
6. **The core challenge** — the one or two genuinely hard ideas this problem tests.
7. **API & protocol design** — the contract between clients and servers.
8. **Data model & storage** — what we persist, where, and why.
9. **High-level architecture** — the boxes and the arrows, with every box justified.
10. **Deep dives** — the mechanisms, including complete Rust reference implementations.
11. **Trade-offs & alternatives** — what we gave up, and what we would do differently.
12. **Wrap-up** — likely follow-up questions, common pitfalls, and an interview checklist.

Conventions used in this book

Three conventions deserve explicit mention.

Terms are defined before they are used. System design is full of overloaded vocabulary — “consistency”, “snapshot”, “version vector” — and interviews are lost when a candidate uses a term they cannot define. Whenever a new concept appears, it is introduced in a **Definition** box before the surrounding text relies on it.

All code is written in Rust. Rust's explicit ownership and type system make concurrency logic — the hardest part of most designs — precise and auditable. The listings are not toys: they are faithful, compilable sketches of the real algorithms, with tests.

Numbers are always justified. Every estimate states its assumptions. A number without assumptions is decoration; a number with assumptions is engineering.

How this book grows

This is a living document. New chapters are appended over time, each tackling one new problem. The framework above is fixed, so you always know where to find the requirements, the math, the architecture, and the code — regardless of the problem.

Providence Salumu — September 2026



Designing a Real-Time Collaborative Text Editor

PROBLEM SOURCE

This chapter solves the problem posed in the talk [“12: Design Google Docs / Real Time Text Editor”](#) from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 47 min). The talk walks the problem in mock-interview form — both a high-level design (HLD) and a low-level design (LLD) of the concurrency machinery. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

1.1 The Problem Statement

The interviewer looks up and says:

“Design a real-time collaborative text editor — something like Google Docs. Multiple people should be able to edit the same document at the same time and see each other’s changes as they happen.”

This is one of the richest prompts in the system design canon. It looks like a CRUD application — until you realize that its heart is a distributed-state synchronization problem with no trivially correct answer. The prompt deliberately leaves almost everything open: scale, feature scope, consistency guarantees, offline behavior. How you close those gaps is most of the grade.

DEFINITION — High-level design (HLD) / low-level design (LLD)

HLD is the architecture of the whole system: the major services, data stores, and network paths, and the justification for each. *LLD* is the internal mechanics of the interesting components: data structures, algorithms, and protocols. A strong interview performance does HLD first, then drills into LLD for the one or two components that carry the real difficulty — in this problem, the concurrency-control core.

1.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. The series this chapter draws on runs its interviews as a dialogue, and that is exactly how a real interview should feel. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	“Are we building plain text editing, or rich text — formatting, embedded images, tables?”

Speaker	Dialogue
Interviewer	“Plain text is fine. Assume a document is an ordered sequence of characters.”
Candidate	“How many collaborators can edit one document simultaneously?”
Interviewer	“Up to 50 concurrent editors per document.”
Candidate	“And the total scale — how many users and documents are we designing for?”
Interviewer	“10 million daily active users. Documents themselves can be up to a few hundred kilobytes of text.”
Candidate	“Do users need to see each other’s cursors and selections live — presence?”
Interviewer	“Yes, cursors and names, like Google Docs shows.”
Candidate	“Is offline editing in scope, or can we assume clients are connected while editing?”
Interviewer	“Assume connected for the core design; if time remains, sketch how offline would work.”
Candidate	“Do we need version history — the ability to view and restore older revisions?”
Interviewer	“Yes, keep a full history of changes.”

From this exchange we can freeze the scope. Everything later in the chapter traces back to these decisions, so state them explicitly and get a nod from the interviewer before moving on.

AGREED SCOPE

Plain-text documents up to 500 KB; up to **50 concurrent editors per document**; **10M daily active users**; live cursors and presence; full version history; connected editing for the core design (offline sketched as an extension).

INTERVIEW TIP — Negotiate scope in both directions

Candidates think clarifying questions only **shrink** scope. They also **expand** it deliberately: asking “do we need presence?” signals that you know the feature exists and costs something. Offer it, let the interviewer decide, and note the cost either way.

1.3 Functional Requirements

DEFINITION — Functional requirement (FR)

A statement of what the system must **do** — an observable behavior a user can verify, such as “a user can create a document.” Functional requirements define the feature set; they say nothing about how fast, how available, or how consistent the system must be. Those qualities are *non-functional requirements*, defined next.

Our scoped functional requirements:

1. **FR-1 — Document management.** Users can create, rename, open, and delete documents.
2. **FR-2 — Real-time collaborative editing.** Multiple users can edit one document concurrently; every connected editor sees every other editor’s changes within a fraction of a second.
3. **FR-3 — Presence.** Each editor sees who else is in the document, with a live cursor (and selection) per collaborator, labeled with their name.

4. **FR-4 — Persistence.** A document is never lost because a client disconnects; reopening it later yields its latest committed state.
5. **FR-5 — Version history.** Users can browse the full history of a document and view or restore any earlier revision.
6. **FR-6 — Sharing & access control.** A document has an owner; the owner grants view or edit access to others via a shareable link or explicit invite.

Out of scope (say so explicitly): rich text, comments and suggestions, end-to-end encryption, and offline-first editing as a core flow.

1.4 Non-Functional Requirements

DEFINITION — Non-functional requirement (NFR)

A statement of how **well** the system must perform its functions: scale, latency, availability, durability, consistency. NFRs are where design decisions actually come from — two systems with identical functional requirements (a chat app and a payment ledger) can have opposite architectures because their NFRs differ.

Three qualities dominate this problem, and they deserve precise vocabulary:

DEFINITION — Latency

The time between a cause and its observable effect. Two latencies matter here: *edit latency* (my keystroke appears in my own document — must feel instant, under 16 ms, so it must be applied locally without a network round trip) and *propagation latency* (my keystroke appears on a collaborator's screen — our target: under 150 ms for users in the same region).

DEFINITION — Availability

The fraction of time the system is able to serve requests. Editing must remain available even when individual servers fail, so no single machine may be indispensable. We will aim for the classic “three nines” ($99.9\% \approx 8.8$ hours of downtime per year) for the editing path.

DEFINITION — Consistency / convergence

In a system where many actors mutate shared state concurrently, *consistency* describes what guarantees observers get about the state they see. For a collaborative editor the crucial guarantee is **convergence**: once the same set of edits has been delivered to every replica, every replica must hold the **identical document**. If two users could permanently end up looking at different text, the product is broken no matter how fast it is. How convergence is achieved — without locking the document — is the core challenge of this chapter and gets its own section (Section 1.6).

The remaining NFRs, stated as targets:

Quality	Target
Propagation latency	p50 < 150 ms within a region; p95 < 400 ms cross-region
Edit (local echo) latency	< 16 ms — edits apply locally, synchronously, before any server contact
Editors per document	Up to 50 concurrent, no degradation
System scale	10M DAU; 1M simultaneous connections at peak

Quality	Target
Durability	No committed edit is ever lost (history is the product's memory)
Availability	99.9% for editing; reading a document should survive almost anything

KEY INSIGHT — Consistency is the NFR that shapes this design

Most interview problems are dominated by throughput (design Twitter) or storage (design Dropbox). This one is dominated by **correctness under concurrency**. The moment two users type at the same time, you need a mathematical answer to “what is the document now?” — Section 1.6 is where the interview is won or lost.

1.5 Back-of-the-Envelope Estimation

DEFINITION — Back-of-the-envelope estimation

Rapid, approximate arithmetic — using round numbers and stated assumptions — that sizes a system **before** designing it. Its purpose is not precision; it is to discover which constraints bite. A design for 500 messages per second and a design for 500,000 messages per second are different systems, and you cannot know which one you owe the interviewer until you count.

DEFINITION — DAU / QPS

DAU (daily active users) counts distinct users active in a day. *QPS* (queries per second) is the request rate a system handles. Interview estimates usually convert DAU into QPS via an activity model (“each user does X, Y times a day”).

Assumptions (state them, write them down, invite correction):

- 10M DAU; each user actively edits on 3 documents per day, 20 minutes per session.
- At peak, 5% of DAU are simultaneously connected: **500k concurrent connections** (we design for 1M to leave headroom).
- An active typist produces 3 keystroke-operations per second in bursts; averaged over a session (reading, thinking, pausing), 0.5 ops/sec per connected user.
- One operation (a character insert/delete plus metadata) $\approx$ 250 bytes on the wire.
- Average document size: 20 KB of text (500 KB worst case per our scope).

Derived numbers:

Quantity	Estimate	How
Peak edit operations	$\approx$ 250k ops/sec	500k conns $\times$ 0.5 ops/s
Edit ingress bandwidth	$\approx$ 60 MB/s	250k ops/s $\times$ 250 B
Operations stored per day	$\approx$ 2.2B	25k ops/s avg $\times$ 86,400 s
Operation log growth	$\approx$ 550 GB/day	2.2B ops $\times$ 250 B
New documents per day	$\approx$ 3M	10M users $\times$ 0.3
Doc-open QPS (REST)	$\approx$ 600 avg / 6k peak	10M $\times$ 5 opens/day $\div$ 86,400, $\times$ 10 peak factor

Quantity	Estimate	How
Connections per WebSocket gateway	50k	commodity servers hold tens of thousands of idle-ish connections
Gateways needed	20–40	1M conns ÷ 25–50k each, plus redundancy

KEY INSIGHT — What the math tells us

Two facts jump out. First, the **system-wide** write rate (hundreds of thousands of ops/sec) is large but utterly **sharded by document**: each document sees at most 50 editors producing maybe 150 ops/sec in a frantic burst — trivial for one thread. Second, the connection tier is where the scale lives: a million mostly-idle WebSocket connections is a capacity problem, not a correctness problem. Conclusion: put the hard correctness logic **per document** (where throughput is tiny) and make the connection tier **stateless and horizontal** (where throughput is huge). This observation drives the entire architecture in Section 1.8.

1.6 The Core Challenge: Concurrent Editing

Everything else in this chapter is standard distributed-systems plumbing. This section is the intellectual center of the problem — and of the source talk.

1.6.1 Why naive approaches fail

Suppose two users, Alice and Bob, are editing the document "GO" — a two-character document: G at index 0, O at index 1. They type **at the same time**:

- **Alice** inserts the character A at index 0 (she is turning "GO" into "AGO").
- **Bob** deletes the character at index 1 (he is turning "GO" into "G").

What should the document be when both edits are delivered? Both users' intentions are clear, and they do not conflict in spirit: the answer is "AG" — Alice's A before the G, and the O gone. Now examine two naive strategies:

Naive strategy 1 — lock the document. Only one editor at a time; everyone else waits. This is **correct** but useless: it defeats the entire purpose of the product. Real-time collaboration means no global mutual exclusion.

DEFINITION — Mutual exclusion / locking

A concurrency-control technique in which a resource may be modified by only one actor at a time; others block until the lock is released. Locks serialize work, guaranteeing correctness at the cost of parallelism — fatal when the resource is a shared document with 50 simultaneous editors.

Naive strategy 2 — last-writer-wins. Let each edit carry a timestamp; the edit with the latest timestamp prevails, earlier conflicting edits are discarded.

DEFINITION — Last-writer-wins (LWW)

A conflict-resolution rule: when two writes to the same datum race, keep the one with the later timestamp and drop the other. Simple and fast, but it **loses data by design** — one of Alice's or Bob's edits would silently vanish. LWW is acceptable for overwritten fields (a profile picture), never for merged text.

Applying LWW to our example: if Bob's delete "wins", Alice's A disappears. If Alice's insert "wins", Bob's delete is lost and the O resurrects. Neither user did anything wrong, yet one of their intentions

was destroyed. And naive **position-based** application is worse still: if Bob's `delete index 1` is applied to Alice's already-updated `"AG0"`, it deletes the `G` instead of the `0`, producing `"AO"` — a document **neither** user intended.

1.6.2 What “correct” means

We need guarantees that can be stated precisely and proven. Three of them:

DEFINITION — Causality

Event A *causally precedes* event B if A happened first and could have influenced B — for example, B is an edit made by a user who had already seen A applied. Two events with no causal order either way are **concurrent**. Alice's and Bob's edits above are concurrent: neither saw the other's. Concurrency — not time — is what creates conflicts; timestamps cannot detect it, which is why Section 1.7 introduces version vectors.

DEFINITION — Convergence

A replica set converges if, once the **same set** of edits has been delivered to every replica, every replica holds the **identical** document — regardless of the order in which the edits arrived. Convergence is a property of the algorithm, not of luck: it must hold for every possible interleaving.

DEFINITION — Intention preservation

The effect of an edit, as observed by its author at the moment they made it, must survive concurrency with other edits. Alice meant “A before G” and Bob meant “0 is gone”; the converged document “AG” honors both. An algorithm can converge and still be bad (converging to the empty string is trivial), so convergence alone is not enough — we want convergence **with** intention preservation.

There are exactly two families of algorithms in wide production use that deliver these properties: **Operational Transformation** and **Conflict-free Replicated Data Types**. A candidate who can define, contrast, and implement one of them owns this interview.

1.6.3 Approach A — Operational Transformation (OT)

DEFINITION — Operational Transformation (OT)

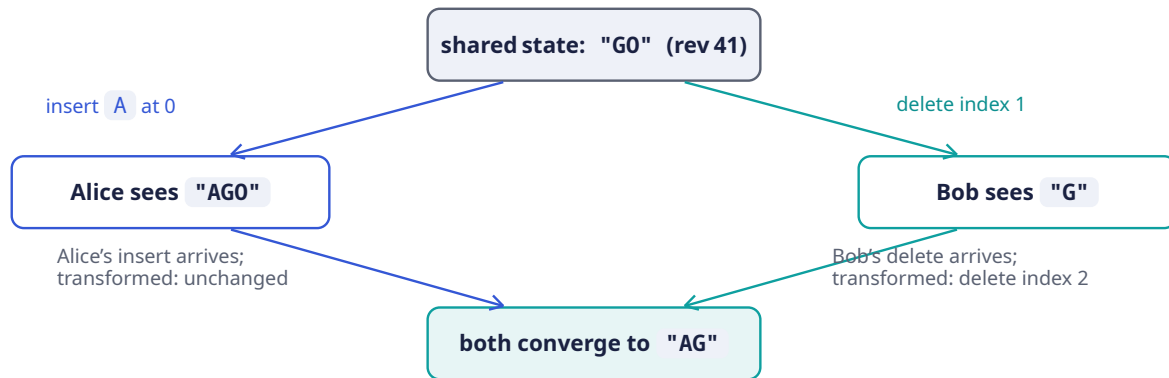
A concurrency-control technique in which every edit is expressed as an **operation** — for plain text, `insert(position, char)` or `delete(position)` — and, when two operations are concurrent, one is **transformed** against the other: its position is adjusted so that it applies correctly to a document the other operation has already modified. OT powers Google Docs. Its lineage: the Jupiter system (1995), then Google Wave, then Docs.

The transformation rules are a small, complete matrix. To apply operation `a` after concurrent operation `b` has already been applied, transform `a` against `b`:

Pair	Rule for transforming <code>a</code> against already-applied <code>b</code>
<code>insert(p_a)</code> vs <code>insert(p_b)</code>	If <code>p_b < p_a</code> , shift <code>a</code> right by 1. If <code>p_b == p_a</code> , break the tie deterministically (e.g., by client ID) so every replica shifts the same way.
<code>insert(p_a)</code> vs <code>delete(p_b)</code>	If <code>p_b < p_a</code> , shift <code>a</code> left by 1. Otherwise unchanged.
<code>delete(p_a)</code> vs <code>insert(p_b)</code>	If <code>p_b <= p_a</code> , shift <code>a</code> right by 1. Otherwise unchanged.

Pair	Rule for transforming a against already-applied b
<code>delete(p_a)</code> vs <code>delete(p_b)</code>	If $p_b < p_a$, shift <code>a</code> left by 1. If $p_b == p_a$, both deletes target the same character: <code>a</code> becomes a no-op.

The whole of OT is this matrix plus a discipline: **every replica applies the same operations in the same total order, transforming as needed**. Our running example, drawn as the classic OT convergence diamond — both paths from the concurrent state must land on the same document:



Walking the diamond: on Alice's side, Bob's `delete(1)` is transformed against her already-applied `insert(0, A)` — since the insert sits at or before the delete position, the delete shifts right to `delete(2)`, which removes the `0` from "AG0" and yields "AG". On Bob's side, Alice's `insert(0, A)` is transformed against his already-applied `delete(1)` — the insert position is before the delete, so it applies unchanged to "G", yielding "AG". Same operations, same converged document, both intentions preserved.

NOTE — Where the total order comes from

OT requires every replica to transform against the **same** concurrent history. The standard way to arrange that is a **single sequencer** per document: the server assigns every operation the next integer **revision** (42, 43, 44, ...), and that assignment is the total order. Clients send operations tagged with the revision they were based on; the server transforms them across any operations committed in between. We make this precise in the deep dives (Sections 1.9 and 1.10).

1.6.4 Approach B — Conflict-free Replicated Data Types (CRDTs)

DEFINITION — Eventual consistency

A consistency model in which replicas that have stopped receiving new writes will **eventually** reach the same state. It promises convergence but says nothing about **when**, and nothing about what intermediate states look like.

DEFINITION — Conflict-free Replicated Data Type (CRDT)

A data structure whose concurrent updates are merged by rules that are commutative, associative, and idempotent — so replicas that apply the same updates in **any** order, even with duplicates, provably converge. This upgraded guarantee is called **strong eventual consistency**: convergence as soon as updates are delivered, with no central ordering and no transformation step.

For text, the canonical CRDT trick is to stop addressing characters by **integer positions** (which shift under concurrency — the very problem OT transforms away) and instead give every character a

unique, immutable, orderable identifier. In the RGA family of algorithms, an insert is recorded as “insert this new character, with a fresh globally-unique ID, immediately after character ID X.” Concurrent inserts after the same character interleave deterministically by ID order; deletes merely mark a character as a tombstone (present in structure, invisible in text). Because identifiers never shift, no transformation is ever needed — merging is just set union.

1.6.5 Choosing between them

Criterion	OT	CRDT
Message size	Tiny: position + char	Larger: globally-unique IDs per character
Memory / metadata	None beyond the text	ID + tombstone per character; tombstone growth needs garbage collection
Needs central sequencer	Yes (classic form)	No — tolerates any delivery order
Algorithmic subtlety	Transformation matrix + history; easy to get subtly wrong	Merge rules; conceptually cleaner, harder to keep compact
Offline / peer-to-peer	Awkward (needs the ordering authority)	Natural fit
Used in production by	Google Docs, older Etherpad	Figma (variants), many local-first tools

Our decision: classic OT behind a per-document sequencer — the architecture the source talk describes, and the one Google Docs runs on. The sequencer is not a bottleneck (Section 1.5 showed a document peaks at 150 ops/sec) and it collapses the hard problem into “transform against a linear log,” which we can implement exactly. We keep version vectors (next section) for reconnect and catch-up.

INTERVIEW TIP — Name the trade-off out loud

Saying “I’ll use OT because Google Docs does” earns no points. Saying “I’ll use OT with a per-document sequencer, accepting a single point of ordering per document — which I mitigate with failover — in exchange for small messages and a linear history that makes version history trivial” is a senior answer. Every choice in this chapter is presented as benefit-purchased-at-cost.

1.7 Version Vectors: Tracking What Each Replica Has Seen

The OT diamond handles the moment of concurrency. But a real system must also answer a book-keeping question constantly: **which operations has this client already received?** On reconnect, on history fetch, on offline sync, the server and client must compare “what you have seen” against “what exists.” Timestamps cannot express this (clock skew makes “when” unreliable across machines). We need a **logical** notion of time.

DEFINITION — Logical clock / Lamport timestamp

A counter, maintained by each participant, that ticks on every local event and is carried with every message; receivers merge it by taking the maximum and ticking on. Logical clocks order events by causality rather than by wall-clock time, which is exactly what distributed state needs — wall clocks on different machines cannot be trusted to agree.

DEFINITION — Version vector

A map from participant ID to a counter: {Alice: 3, Bob: 1} means “I have seen 3 events from Alice and 1 from Bob.” Two version vectors compare pointwise:

- V_1 **dominates** V_2 (V_2 happened-before V_1) if every entry of V_1 is $\geq$ the corresponding entry of V_2 , and at least one is strictly greater — V_1 has seen everything V_2 has, and more.
- If neither dominates, the two states are **concurrent** — each has seen something the other has not.

A version vector is a Lamport clock generalized to many writers: it captures exactly which causal history a replica has absorbed.

In our design the server already assigns a single integer revision per document (the sequencer), so the **steady-state** sync marker is just `rev = 47`. Version vectors earn their place at the edges of the design: when a client reconnects after a network blip during which a **snapshot** was taken, when replicas in different regions compare state, and in the offline extension. The comparison logic is the same everywhere: if the server’s knowledge dominates the client’s, send exactly the difference.

COMMON PITFALL — The snapshot / version-vector trap

The source talk’s wry description — a client “never received those writes on your local copy since your version vector was more up to date than the document snapshot” — points at a real and common bug. Scenario: a snapshot of the document is written to one store, but the version metadata recording **which revisions the snapshot covers** is written to another, and the two writes are not atomic. A reconnecting client compares its version vector against **stale** snapshot metadata, concludes it is up to date, and never receives the missing operations. The cure is atomicity between a snapshot and its version metadata — Section 1.11 shows how, and defines the two-phase commit protocol the talk name- drops for exactly this purpose.

1.8 API & Protocol Design

The system has two distinct planes, and separating them is itself a design decision worth naming:

- The **control plane** — create/rename/share/open documents, fetch history. These are classic request/response operations; plain HTTPS REST is correct and simple.
- The **data plane** — the live editing session: join, send operations, receive operations, presence. This needs a persistent, bidirectional, low-latency channel per client.

DEFINITION — REST

An API style over HTTP in which resources (nouns: `/documents/{id}`) are manipulated with a fixed set of methods (verbs: `GET`, `POST`, `PATCH`, `DELETE`). Stateless by design: each request carries everything the server needs. Ideal for our control plane.

DEFINITION — WebSocket

A protocol (RFC 6455) that upgrades an HTTP connection into a long-lived, full-duplex channel: after an initial handshake, **either** side may send a message at **any** time, with a few bytes of framing overhead. This is what makes sub-150 ms propagation possible — the server can push an edit the instant it is committed, with no client polling and no new connection setup.

Why not the alternatives?

Mechanism	Behavior	Verdict
Short polling	Client re-asks “anything new?” every few seconds	Latency of seconds; wasted requests — rejected
Long polling	Server holds the request open until data exists	Near-real-time but a new HTTP request per message burst — heavy at our scale
Server-Sent Events	Server → client push over HTTP; client sends via separate requests	Workable, but asymmetric; WebSocket is cleaner for a two-way edit stream
WebSocket	One persistent, bidirectional connection	Chosen for the data plane

1.8.1 Control-plane endpoints

Method	Path	Purpose
POST	/documents	Create a document; returns its <code>doc_id</code>
GET	/documents/{id}	Fetch metadata + the latest committed revision number
PATCH	/documents/{id}	Rename, change settings
DELETE	/documents/{id}	Delete (owner only)
POST	/documents/{id}/share	Grant view/edit access to a user or link
GET	/documents/{id}/history?from=&to=	List revisions for the version-history UI
GET	/documents/{id}/snapshot?rev=	Fetch the document as of any revision (view/restore)

1.8.2 Data-plane protocol (WebSocket messages)

After the REST GET, the client opens a WebSocket to the gateway and speaks a small JSON protocol. Every message has a `type`; the important ones:

Message	Direction	Payload & semantics
join	client → server	{doc_id, auth_token, last_seen_rev} — enter the session; last_seen_rev enables catch-up
joined	server → client	{doc_id, rev, snapshot, collaborators[]} — current state, possibly after catch-up
op	client → server	{doc_id, base_rev, op, client_seq} — one edit, based on base_rev, idempotent via client_seq
ack	server → client	{client_seq, rev} — “your edit is committed as revision rev”
op (broadcast)	server → client	{rev, author, op} — a committed edit to apply (transform locally if needed)
presence	both	{cursor, selection, name} — ephemeral, never persisted
sync	server → client	{rev} / snapshot — sent when the client is too far behind to patch with ops

Example committed-operation broadcast:

```
{
  "type": "op",
  "doc_id": "d_8f31c2",
  "rev": 47,
  "author": "alice@example.com",
  "op": { "kind": "insert", "pos": 0, "ch": "A" }
}
```

Two protocol details deserve emphasis because interviewers probe them. First, `base_rev` is how the server detects concurrency: if the document is at revision 52 and your operation says `base_rev: 49`, the server transforms it across revisions 50–52 before committing (Section 1.10). Second, `client_seq` is an **idempotency key**:

DEFINITION — Idempotency / idempotency key

An operation is *idempotent* if performing it twice has the same effect as performing it once. Networks retry; clients reconnect and resend. A unique client-supplied key per operation (here, `(client_id, client_seq)`) lets the server recognize a duplicate delivery and answer with the original result instead of applying the edit twice — the difference between at-least-once **delivery** and effectively exactly-once **effect**.

1.9 Data Model & Storage

Four persistent entities, each with a clear job:

Entity	Contents	Store
Document	<code>doc_id</code> , owner, title, ACL, current <code>rev</code>	Metadata DB (relational or document store; sharded by <code>doc_id</code>)
Operation	<code>doc_id</code> , <code>rev</code> , author, transformed op payload, <code>client_seq</code> , timestamp	Operation log — append-only store, ordered by <code>rev</code> per document
Snapshot	<code>doc_id</code> , <code>base_rev</code> , full document text	Blob/object store; pointer + <code>base_rev</code> in metadata DB
Session	who is connected, cursors	Ephemeral presence cache — not persisted

The two storage ideas that interviewers reward:

DEFINITION — Operation log (event log)

Storing every change as an immutable, ordered, append-only record instead of overwriting state. Current state is always derivable by replaying the log. For us the log **is** the product's memory: version history (FR-5) is free, auditing is free, and recovery is replay. Append-only logs are also the fastest possible write pattern — sequential appends, no random updates.

DEFINITION — Snapshot

A materialized copy of the full document as of a specific revision. Replaying 2 billion operations to open a document is absurd; instead store a snapshot every N operations (say 500) plus the handful

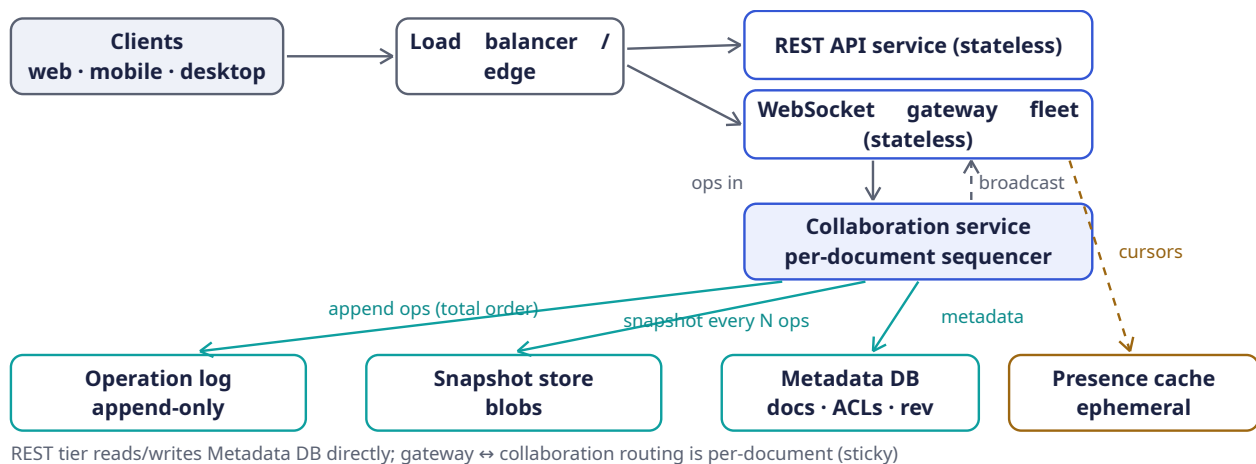
of operations since, and reconstruct state as `snapshot + tail of the log`. Snapshots are a read optimization over the log, never the source of truth — the log is.

NOTE — Why a sharded relational/document store + blob store, not one database

The metadata (small, hot, transactional) wants indexes and conditional updates; snapshots (large, immutable, cold) want cheap object storage; the operation log (append-only, ordered per key) wants a log-structured store. Choosing one engine per access pattern — and **saying why** — is the answer this section exists to elicit. In an interview, name concrete instincts: PostgreSQL-class for metadata, an S3-class blob store for snapshots, a Kafka/Cassandra-class append store for the log — while stressing the **properties** matter more than the brands.

1.10 High-Level Architecture

The estimation (Section 1.5) gave us the organizing principle: **a thin, stateless, horizontally scaled connection tier in front of a small, stateful, correctness-critical core**. Here is the whole system at a glance:



Component responsibilities, and — more importantly — the reason each exists:

Component	Responsibility	Why it is shaped this way
WebSocket gateway	Hold 50k client connections each; terminate TLS; forward messages	Stateless: any client can reconnect to any gateway. Scale by adding boxes — this tier absorbs the 1M-connection problem
Collaboration service	Owns each active document: sequences ops, transforms, commits, broadcasts	Stateful but sharded by document : a document's whole session lives on one node, so its sequencer is just an in-memory counter. Throughput per doc is tiny (Section 1.5), so one node hosts thousands of docs
Operation log	Durable, ordered, append-only record per document	Source of truth; powers history (FR-5), replay, recovery

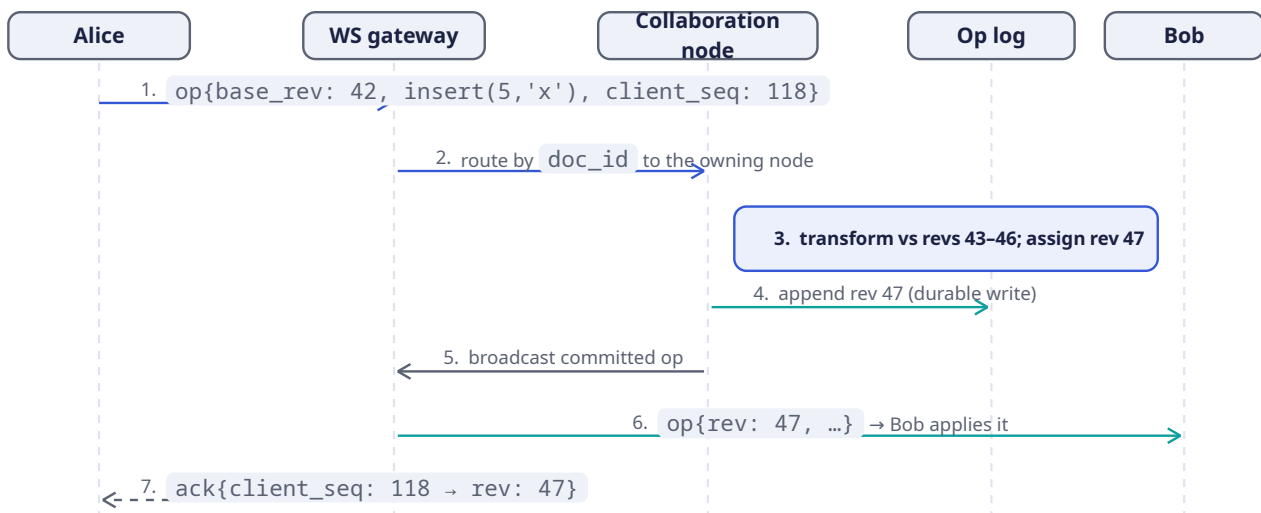
Component	Responsibility	Why it is shaped this way
Snapshot store	Immutable document snapshots every N ops	Makes open/reconnect $O(1)$ instead of $O(\text{history})$
Metadata DB	Documents, ACLs, current revision counters	Small, hot, transactional reads/writes
Presence cache	Live cursors and collaborator lists	Ephemeral by nature — losing it costs nothing; it rebuilds in seconds
REST API service	Control plane (Section 1.8)	Stateless request/response work; ordinary horizontal scaling

KEY INSIGHT — The one routing rule that makes it all work

All traffic for a given document flows through the single collaboration node that owns it. Gateways look up `doc_id` → owner (a cheap routing-table read) and forward. Because every operation on a document passes through one owner, the owner's revision counter is a **total order** — which is exactly what OT needs (Section 1.6). Ownership can migrate on failure; the invariant is that there is exactly one owner at a time.

1.11 Deep Dive: The Edit Pipeline, Step by Step

Follow one keystroke from Alice's keyboard to Bob's screen. The document `d_8f31c2` is at revision 46; Alice believes it is at revision 42 (she has not yet received four operations from other editors).



In words:

- Send.** Alice's keystroke is applied to her local document **immediately** (local echo — this is how edit latency stays under 16 ms) and sent as `op{base_rev: 42, ...}` with her next `client_seq`.
- Route.** The gateway does no correctness work; it reads `doc_id`, finds the owning collaboration node, forwards.
- Sequence & transform.** The owner sees `base_rev: 42` but the document is at 46: Alice's operation is **concurrent** with revisions 43–46. It transforms her operation against each of them, in order (Section 1.13 implements this loop), then stamps it with the next revision, 47.

4. **Commit.** The transformed operation is appended to the operation log. Once the append is durable, the edit is **committed** — it can never be reordered or lost.
5. **Broadcast.** The committed operation goes to the gateways of every connected editor of the document.
6. **Apply.** Bob's client is not behind (its `base_rev` matches), so it applies the operation verbatim. If Bob had a **pending** unacknowledged local edit, he would first transform it against this incoming operation — the client runs the same transform matrix as the server, keeping his local echo intact.
7. **Acknowledge.** Alice learns her operation is committed as revision 47; her pending buffer clears. Her screen never flickered — the local echo and the committed result are identical by construction.

COMMON PITFALL — Order matters: commit before broadcast

If you broadcast before the log append is durable, a node crash between the two steps shows users edits that no longer exist — and on recovery, the “committed” history disagrees with what people saw. Durability first, **then** visibility. This is the write-path equivalent of “don’t acknowledge what you can’t guarantee.”

1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson

Two housekeeping jobs remain: making **open/reconnect** fast, and keeping a snapshot and its metadata **atomic**. Both live at the seam between the operation log and the snapshot store.

1.12.1 Catch-up: what to send a rejoining client

When a client sends `join{last_seen_rev}`, the owner compares it to the current revision `R`:

- **Close behind** ($R - \text{last_seen_rev} \leq 500$): send the missing operations from the log. The client transforms any pending local edits across them and is current.
- **Far behind** (offline for hours): replaying thousands of ops is slower than sending state. Send `latest snapshot + ops since the snapshot's base_rev`.
- **Offline edits** (the extension): the client presents its **version vector** (Section 1.7) instead of a scalar revision; the server computes the set difference — ops the client missed — and the client rebases its offline ops on top. Vectors, not timestamps, because they compare **causal** histories.

1.12.2 The atomicity requirement

A snapshot is useless without its metadata (`base_rev`: which revisions it covers), and dangerous with **stale** metadata — that is exactly the failure the source talk jokes about: a client whose version vector is “more up to date than the document snapshot” concludes it needs nothing and **silently misses writes**. Two writes must succeed or fail **together**: the snapshot blob, and the `{snapshot_pointer, base_rev}` record in the metadata DB.

DEFINITION — Two-phase commit (2PC)

A protocol for committing one atomic operation across **multiple** stores. Phase 1 (*prepare*): a coordinator asks every participant “can you commit?” Each participant does everything short of committing and votes yes/no. Phase 2 (*commit*): if all voted yes, the coordinator tells everyone to commit; if any voted no, everyone aborts. 2PC guarantees atomicity but is **blocking** (a crashed coordinator can leave participants waiting) and slow (two round trips) — so we use the minimal version of the idea, not the maximal one.

In practice, prefer the cheapest construction that delivers atomicity:

1. **One store, one transaction.** If the snapshot blob and its metadata live in the same database, a single transaction suffices — no 2PC at all.

2. **Immutable blob + conditional pointer update** (our choice). Write the snapshot to the blob store under a content-addressed name; only when that write is durable, update the metadata row with a conditional compare-and-swap (`UPDATE ... WHERE base_rev = <old>`). A crash between the steps leaves an **orphan blob** — garbage-collectable, never incorrect. Atomicity without a coordinator.
3. **Full 2PC** is reserved for when two genuinely independent systems must commit together — worth naming in the interview precisely so you can argue you rarely need it.

1.13 Deep Dive: Rust Reference Implementation

The concurrency core is small enough to show in full. Three pieces: the wire protocol types, the OT transform matrix with tests, and the version vector.

1.13.1 Protocol types

The data-plane messages from Section 1.8, as Rust types (serde for JSON):

```
use serde::{Deserialize, Serialize};

/// One edit to a plain-text document.
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum Op {
    Insert { pos: usize, ch: char },
    Delete { pos: usize },
}

/// Data-plane messages (client → server), serialized as JSON.
#[derive(Clone, Debug, Serialize, Deserialize)]
#[serde(tag = "type", rename_all = "snake_case")]
pub enum Msg {
    Join { doc_id: String, auth: String, last_seen_rev: u64 },
    Joined { doc_id: String, rev: u64, snapshot: String },
    Op { doc_id: String, base_rev: u64, client_seq: u64, op: Op },
    Ack { client_seq: u64, rev: u64 },
    /// Server → clients: a committed operation.
    Bcast { doc_id: String, rev: u64, author: String, op: Op },
}
```

The `#[serde(tag = "type")]` attribute makes the JSON look exactly like the protocol table: a `type` field distinguishes the variants.

1.13.2 The OT transform matrix

This is Section 1.6's table, made executable. `transform(a, b, a_wins_ties)` returns the operation `a` adjusted to apply **after** the concurrent operation `b` has been applied:

```
use crate::Op;

/// Transform `a` so it applies correctly *after* concurrent op `b`.
/// `a_wins_ties` breaks same-position insert/insert ties identically
/// on every replica (e.g., by client id) — determinism is what matters.
pub fn transform(a: &Op, b: &Op, a_wins_ties: bool) -> Option<Op> {
    match (a, b) {
        (Op::Insert { pos: pa, ch: ca }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa > *pb || (*pa == *pb && !a_wins_ties) { pa + 1 } else
            { *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
        (Op::Insert { pos: pa, ch: ca }, Op::Delete { pos: pb }) => {
            let shifted = if *pa > *pb { pa - 1 } else { *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
    }
}
```

```

    }
    (Op::Delete { pos: pa }, Op::Insert { pos: pb, .. }) => {
        let shifted = if *pa >= *pb { pa + 1 } else { *pa };
        Some(Op::Delete { pos: shifted })
    }
    (Op::Delete { pos: pa }, Op::Delete { pos: pb }) => {
        if pa == pb { None } // both deleted the same char
        else { Some(Op::Delete { pos: if *pa > *pb { pa - 1 } else { *pa } }) }
    }
}

/// Apply an operation to a document buffer.
pub fn apply(doc: &mut String, op: &Op) {
    match op {
        Op::Insert { pos, ch } => {
            let byte = doc.char_indices().nth(*pos).map_or(doc.len(), |(i, _)| i);
            doc.insert(byte, *ch);
        }
        Op::Delete { pos } => {
            if let Some((byte, c)) = doc.char_indices().nth(*pos) {
                doc.drain(byte..byte + c.len_utf8());
            }
        }
    }
}

```

Note the discipline the code makes explicit: character positions, not byte offsets (Rust strings are UTF-8; mixing the two is a classic production bug), and `None` for the delete/delete collision — an operation transformed into a no-op.

The convergence diamond from Section 1.6, as a test:

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn go_diamond_converges() {
        let alice_op = Op::Insert { pos: 0, ch: 'A' }; // "G0" -> "AGO"
        let bob_op = Op::Delete { pos: 1 }; // "G0" -> "G"

        // Alice's side: apply hers, transform Bob's against it, apply.
        let mut a_doc = String::from("G0");
        apply(&mut a_doc, &alice_op);
        let bob_at_alice = transform(&bob_op, &alice_op, false).unwrap();
        apply(&mut a_doc, &bob_at_alice);

        // Bob's side: apply his, transform Alice's against it, apply.
        let mut b_doc = String::from("G0");
        apply(&mut b_doc, &bob_op);
        let alice_at_bob = transform(&alice_op, &bob_op, true).unwrap();
        apply(&mut b_doc, &alice_at_bob);

        assert_eq!(a_doc, b_doc); // convergence
        assert_eq!(a_doc, "AG"); // intention preservation
    }
}

```

1.13.3 The server-side session: transforming against history

The owner node keeps a per-document session. When an operation arrives based on an old revision, transform it across everything committed since (step 3 of the pipeline):

```
pub struct DocSession {
    pub rev: u64,
    pub doc: String,
    pub log: Vec<(u64, Op)>, // (revision, committed op); the op log in memory
    pub since_snapshot: usize, // ops committed since last snapshot
}

impl DocSession {
    pub fn commit(&mut self, base_rev: u64, mut op: Op) -> Result<u64, &'static str> {
        if base_rev > self.rev { return Err("base_rev from the future"); }
        // Transform across every revision the client had not seen.
        for (_, past) in self.log.iter().skip(base_rev as usize) {
            op = transform(&op, past, false).ok_or("op became a no-op");
        }
        self.rev += 1;
        apply(&mut self.doc, &op);
        self.log.push((self.rev, op.clone()));
        self.since_snapshot += 1;
        if self.since_snapshot >= 500 { self.snapshot(); } // Section 1.9 policy
        Ok(self.rev) // -> ack + broadcast
    }

    fn snapshot(&mut self) { /* write blob, then conditional pointer update */
        self.since_snapshot = 0;
    }
}
```

This is deliberately unglamorous — a counter and a loop. That is the payoff of the architecture: because **all** operations for a document funnel through one owner, the “distributed consensus” needed here is no consensus at all.

1.13.4 Version vector

Section 1.7’s definition, implemented with its three-way comparison:

```
use std::collections::HashMap;

#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct VersionVector(pub HashMap<String, u64>);

#[derive(Debug, PartialEq, Eq)]
pub enum Ordering { Before, After, Equal, Concurrent }

impl VersionVector {
    pub fn tick(&mut self, who: &str) { *self.0.entry(who.into()).or_insert(0) += 1; }

    /// Pointwise comparison: Before = self is causally dominated by other.
    pub fn compare(&self, other: &Self) -> Ordering {
        let mut less = false;
        let mut greater = false;
        for id in self.0.keys().chain(other.0.keys()) {
            let a = self.0.get(id).copied().unwrap_or(0);
            let b = other.0.get(id).copied().unwrap_or(0);
            less |= a < b;
            greater |= a > b;
        }
    }
}
```

```

    match (less, greater) {
      (false, false) => Ordering::Equal,
      (true, false) => Ordering::Before,
      (false, true) => Ordering::After,
      (true, true) => Ordering::Concurrent,
    }
  }

  /// Element-wise maximum: the merged causal history of two replicas.
  pub fn merge(&mut self, other: &Self) {
    for (id, &n) in &other.0 {
      let e = self.0.entry(id.clone()).or_insert(0);
      *e = (*e).max(n);
    }
  }
}

```

The catch-up decision from Section 1.12 is now three lines: if the client’s vector is `Before` the server’s, send the ops it lacks; if `Equal`, send nothing (and — per the pitfall — verify the **snapshot metadata** said the same thing); if `Concurrent`, the client has offline edits: send the diff and rebase.

1.14 Deep Dive: Presence & Live Cursors

Presence — “who is here, and where is their cursor” — looks trivial and hides one elegant detail: **cursor positions are text positions**, so they must be transformed by the same OT matrix as edits. If Alice’s cursor sits at position 10 and Bob inserts three characters at position 4, Alice’s cursor must render at 13. Clients transform remote cursors against every incoming operation; the server simply relays.

Design rules that keep presence cheap:

- **Never persist it.** Presence flows through the ephemeral presence cache and broadcasts; a client heartbeat every few seconds refreshes it. If presence data is lost, it rebuilds within one heartbeat — this is why the architecture draws it as a dashed, separate path.
- **Don’t sequence it.** Presence messages carry no `rev` and bypass the operation log entirely; a stale cursor for 100 ms is invisible to users, while a stale **edit** is a correctness bug.
- **Throttle it.** Cursor moves fire dozens of events per second per editor; cap updates (e.g., 10/sec/client) — 50 editors × 10 updates = 500 tiny messages/sec, which the same per-document owner trivially fans out.

1.15 Scaling & Sharding

DEFINITION — Sharding (horizontal partitioning)

Splitting data or work across many machines by a key, so that each machine owns a disjoint slice. Sharding is the fundamental answer to “one machine is not enough”: pick the right key and load grows by adding machines rather than by growing them.

Every tier shards by its natural key:

- **Gateways:** stateless, so any consistent routing works; connections balance across the fleet, and reconnects land anywhere.
- **Collaboration nodes:** sharded **by document**. A routing service maps `doc_id` → `owner` with a lease (a time-limited ownership grant the node must renew; on crash the lease expires and ownership

moves — this is how we keep “exactly one sequencer per document” without human intervention).

- **Operation log / snapshots / metadata:** sharded by `doc_id` as well, so all of a document’s data is local to its shard.

KEY INSIGHT — The 50-editor cap is a load-shedding decision

Why can a document never become a “hot shard” that melts its owner? Because we capped concurrency at 50 editors — a product decision with an architectural dividend. A live-blog with a million **writers** would need a different design (tree of sequencers, or CRDTs with no central order). Naming which NFR protects you from which failure mode is exactly the reasoning interviewers listen for.

1.16 Failure Modes & Recovery

A senior answer enumerates what breaks **before** being asked. The playbook:

Failure	Handling
Client disconnects	Its edits are already durable in the log (commit-before-broadcast). On reconnect: <code>join</code> with <code>last_seen_rev</code> , catch up per Section 1.12. Presence entry expires on its own.
WebSocket gateway dies	Clients reconnect to any other gateway (they are stateless). In-flight unacked ops are resent and deduplicated by <code>client_seq</code> (idempotency).
Collaboration node dies	Lease expires; routing service assigns a new owner, which rebuilds the session state as <code>latest snapshot + log tail</code> and resumes sequencing. Clients see a reconnect, not data loss.
Duplicate message delivery	Impossible to prevent on real networks — so we make it harmless: <code>client_seq</code> idempotency on the write path, revision numbers on the read path (already have rev 47? ignore the re-broadcast).
Op-log replica loss	The log is replicated (3× is standard); a lost replica is replaced from its peers.
Network partition	A client that cannot reach the server keeps editing locally and rebases on reconnect (version-vector diff, Section 1.12) — or shows “offline, changes will sync” if we choose availability over freshness. A partitioned server-side minority refuses ownership transfers: correctness beats liveness there.

1.17 Trade-offs & Alternatives

The design is a bundle of purchased benefits and accepted costs. Saying the costs out loud is what distinguishes a design from a wish list:

Decision	Benefit purchased	Cost accepted
Per-document sequencer + OT	Small messages, total order, free version history, simple reasoning	Single point of ordering per doc (mitigated by lease failover, seconds of unavailability on crash);

Decision	Benefit purchased	Cost accepted
		OT matrix must be exactly right
CRDT instead	No sequencer; offline and peer-to-peer for free	Per-character IDs and tombstones; heavier storage and GC; harder to bolt on a clean linear history
WebSocket data plane	Sub-150 ms push; bidirectional	1M long-lived connections to manage; load balancers must tolerate them
Op log as source of truth	History, audit, replay, recovery all free	Storage growth (550 GB/day at our scale) and the snapshot machinery to keep reads fast
Single-region write ownership per doc	No cross-region consensus on the hot path	Editors far from the owner's region see higher propagation latency

NOTE — PACELC, briefly

A refinement of the CAP theorem: **if** there is a Partition, choose Availability or Consistency; **else** (normal operation), choose Latency or Consistency. Our design picks **consistency** on both branches for the edit path (single owner, total order) and **availability/latency** for presence (ephemeral, best-effort) — different subsystems may legitimately make different PACELC choices.

1.18 Observability & SLOs

DEFINITION — SLI / SLO

A *Service Level Indicator* is a measurable quantity (e.g., propagation latency at p95). A *Service Level Objective* is its target (“p95 < 400 ms”). SLOs turn “the system feels slow” into an engineering signal with an error budget.

What we instrument, at minimum: per-document ops/sec and transform rates; end-to-end propagation latency (client-side measured, tagged by region); ack latency; WebSocket connections and message rates per gateway; catch-up size distribution (reconnect storms reveal themselves here);

`current_rev - snapshot.lag ( snapshot.base_rev );` lease-failover count. Alerts fire on SLO burn, not on individual machine failures — the failure table above exists precisely so that single failures are non-events.

1.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- “*Rich text?*” — Operations gain attributes (bold ranges, styles); the transform matrix grows cases, the architecture does not change.
- “*Comments / suggestions?*” — Anchored to character ranges that transform like cursors; stored in metadata DB, off the hot path.

- “*End-to-end encryption?*” — Server cannot read content, so server-side OT is impossible; this pushes you toward client-side CRDTs. A great example of a requirement that **re-architects** the core.
- “*5,000 simultaneous editors?*” — Our 50-editor cap is what licenses the single sequencer; beyond it, shard the document itself or move to CRDTs.
- “*Undo?*” — Per-user inverse operations transformed through the log — easy to describe, subtly hard to make intuitive; mention it, don’t build it live.

If you remember five things:

1. The product is a distributed-state synchronizer wearing a text editor’s clothes.
2. Convergence + intention preservation are the correctness bar; LWW and locks both fail it.
3. A per-document total order (sequencer) reduces “distributed consensus” to a counter and a transform loop.
4. Commit durably, then broadcast — never show a user an edit you cannot guarantee.
5. Snapshots and their version metadata commit atomically, or clients silently miss writes (Section 1.12).

1.20 Summary & Further Reading

We designed a real-time collaborative plain-text editor for 10M DAU: a stateless WebSocket gateway tier holding a million connections; a collaboration service that owns each document and imposes a total order on its operations; operational transformation to merge concurrent edits; an append-only operation log as the source of truth with periodic snapshots; version vectors for catch-up and offline diffing; and idempotent protocols that survive the network’s misbehavior.

Primary source for this chapter:

- “[12: Design Google Docs/Real Time Text Editor](#)” — [Systems Design Interview Questions With Ex-Google SWE \(Jordan has no life\)](#) — the mock-interview walkthrough this chapter expands.

Foundations worth reading:

- Ellis & Gibbs, *Concurrency Control in Groupware Systems* (1989) — the original OT paper.
- Nichols et al., *Jupiter: high-latency collaboration* (1995) — OT with a central server, the direct ancestor of our design.
- Shapiro et al., *Conflict-free Replicated Data Types* (2011) — the CRDT formulation.
- Google Wave’s OT whitepapers, and Figma’s engineering blog on multiplayer — production war stories for both families.

1.21 Chapter 1 Glossary

A one-glance index of every term this chapter defined. Later chapters assume it.

Term	Meaning in one line
System design interview	Open-ended architecture design under a 45–60 minute clock
HLD / LLD	Whole-system architecture / internal mechanics of the hard components
Functional requirement	What the system must do
Non-functional requirement	How well it must do it: scale, latency, availability, consistency
DAU / QPS	Daily active users / queries per second
Back-of-the-envelope estimation	Approximate arithmetic that sizes the problem before designing

Term	Meaning in one line
Latency	Cause → observable effect; here: local echo < 16 ms, propagation < 150 ms
Availability	Fraction of time the system serves requests
Mutual exclusion (lock)	One writer at a time — correct, but kills real-time collaboration
Last-writer-wins	Latest timestamp overwrites — simple, but silently loses edits
Causality / concurrency	“Could A have influenced B?” If neither way: concurrent
Convergence	Same delivered edits ⇒ identical document on every replica
Intention preservation	Each author’s observed edit effect survives concurrency
Operational Transformation	Adjust concurrent operations’ positions so all replicas converge
CRDT	Data type whose merge rules converge without a central order
(Strong) eventual consistency	Replicas converge (immediately upon delivery, for SEC)
Logical clock / Lamport	Causality-tracking counters instead of wall-clock time
Version vector	Per-participant counters recording exactly what a replica has seen
REST	Stateless resource-oriented HTTPS API — our control plane
WebSocket	Persistent full-duplex client↔server channel — our data plane
Idempotency key	Client-supplied unique key making retried operations safe
Operation log	Immutable append-only record of all edits; the source of truth
Snapshot	Materialized document at a revision; read optimization over the log
Two-phase commit	Prepare-then-commit protocol for atomic writes across stores
Sharding	Partitioning work/data by key across machines — here, by document
Presence	Ephemeral who’s-here-and-where state; never persisted
SLI / SLO	A measured indicator / its target, defining “healthy”

Designing a Maps & Navigation Service

PROBLEM SOURCE

This chapter solves the problem posed in the talk “[13: Google Maps](#)” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 35 min). The talk designs a mapping and navigation product: how the map itself is served, how the road network becomes a graph, how routes and ETAs are computed, and how live traffic feeds back into both. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

2.1 The Problem Statement

The interviewer looks up and says:

“Design Google Maps. Users should be able to look at a map, search for places, and get directions from A to B — with an estimated time of arrival that accounts for current traffic.”

Where Chapter 1’s problem hid its difficulty behind a familiar CRUD interface, this one hides its difficulty behind a familiar *picture*. A map feels like content — something you serve, like an image. Directions feel like a lookup — something you query, like a database. Neither is true. The map is a planet-sized rendering problem, directions are a graph algorithm running at planetary scale, and “current traffic” means the system’s answers change **under you every few minutes**, driven by a firehose of GPS signals from millions of moving phones. Three genuinely hard subsystems — geospatial serving, large-scale graph search, and real-time stream processing — share one interview.

DEFINITION — ETA (estimated time of arrival)

The system’s prediction of how long a journey from A to B will take, usually expressed as a duration or an arrival clock time. An ETA is a **prediction**, not a measurement: it must be computed before the journey happens, from a model of the road network plus everything currently known about conditions on it. ETA accuracy is the quality bar this whole chapter is judged against — Section 2.4 makes it a formal requirement and Section 2.18 shows how to measure it.

2.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. Google Maps is a dozen products wearing one icon, so the first job is to shrink it. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	“The full product is enormous — map rendering, place search, directions, turn-by-turn navigation, live traffic, Street View, satellite imagery, transit schedules, offline maps. Which parts are we designing?”
Interviewer	“Focus on five: render the map, search for places, compute directions with an ETA, show live traffic, and run a turn-by-turn navigation session. Skip Street View, satellite, transit, and offline.”
Candidate	“What scale — how many users, and how many actively navigating at once?”
Interviewer	“One billion monthly users, a few hundred million daily. Assume up to 15 million people in an active navigation session at peak.”
Candidate	“Latency expectations?”
Interviewer	“The map must feel instant while panning. A route request should come back within a second or two.”
Candidate	“How accurate does the ETA need to be?”
Interviewer	“Within about ten percent of actual travel time on typical trips.”
Candidate	“Do drivers’ phones report GPS positions back to us during navigation? That would be our live traffic signal.”
Interviewer	“Yes — anonymized location updates, with user consent, every few seconds while navigating.”
Candidate	“Availability target? People use this while driving; a dead navigation app mid-highway is a safety problem.”
Interviewer	“Four nines for the navigation path.”

AGREED SCOPE

Five features: **map rendering**, **place search**, **directions with ETA**, **live traffic**, **turn-by-turn navigation**. Scale: **1B MAU**, **200M DAU**, **15M concurrent navigation sessions** at peak. Latency: map feels instant, routes within 1–2 s. ETA accurate to $\pm 10\%$. Navigation availability **99.99%**. Phones send anonymized GPS updates every few seconds while navigating.

INTERVIEW TIP — Ask where the data comes from

Most candidates ask about users and features; few ask “**what data do we get to see?**” That single question unlocks this entire problem: the answer (GPS updates from phones) is what makes live traffic possible at all. In a real interview, the questions that reveal **inputs** are worth as much as the questions that reveal **scale**.

2.3 Functional Requirements

Chapter 1 defined functional and non-functional requirements; recall that FRs are what the system must **do**. Our scoped functional requirements:

1. **FR-1 — Map rendering.** The user can view an interactive map of the world and pan and zoom it smoothly, from a whole-planet view down to individual streets.
2. **FR-2 — Place search.** The user can find places by name, address, or category (“coffee near me”) and see them on the map.

3. **FR-3 — Directions.** The user can request a route between two points and receives a good route: distance, step-by-step instructions, and a path drawn on the map.
4. **FR-4 — ETA.** Every route comes with a travel-time estimate that reflects **current** conditions, not just speed limits.
5. **FR-5 — Turn-by-turn navigation.** The user can start a navigation session and receives timely spoken/visual instructions, a live ETA that updates en route, and an automatic new route if they stray from the planned path.
6. **FR-6 — Traffic overlay.** The map shows current congestion (green/yellow/red) on roads, fresh to within a few minutes.

Out of scope, stated explicitly: Street View and satellite imagery, public transit timetables, offline maps, business-owner tooling, and social features.

2.4 Non-Functional Requirements

DEFINITION — Freshness

The age of the information behind an answer, measured from when reality changed to when the system's output reflects it. A traffic overlay that shows speeds from an hour ago is **stale**; our target is that any road's displayed condition reflects measurements from the last few minutes. Freshness is the NFR that makes this problem a **streaming** system rather than a static one.

Four qualities dominate, stated as targets:

Quality	Target
Map latency	Tiles visible within 100 ms while panning (p95); panning must never stutter
Route latency	Directions within 1 s for typical trips; 2 s for cross-country
ETA accuracy	Within $\pm 10\%$ of realized travel time on typical trips
Traffic freshness	Displayed speeds reflect the last 2–5 minutes of reality
Availability	99.99% on the navigation path (4.4 s of allowed downtime per day); 99.9% elsewhere
Scale	1B MAU; 200M DAU; 15M concurrent navigation sessions at peak

KEY INSIGHT — Different subsystems, different NFRs

Notice that “the system” has no single latency or availability number. Tile serving is a massive, cacheable **read** workload where 100 ms matters and a stale tile is harmless. Routing is a **compute** workload where one second is generous but the algorithm must be exact. The traffic pipeline is a **write** workload where a few minutes of lag degrade quality, not correctness. Saying “it depends which plane you mean” — and then naming the planes — is a senior answer. Chapter 1 drew the same lesson with its control plane / data plane split.

2.5 Back-of-the-Envelope Estimation

Chapter 1 defined back-of-the-envelope estimation; the discipline is identical — state assumptions, write them down, invite correction.

Assumptions:

- 1B MAU, 200M DAU. Each daily user views 60 map tiles worth of panning, searches 2 places, and requests 3 routes per day.

- 15M concurrent navigation sessions at peak; each phone uploads one GPS update every 5 seconds while navigating; one update $\approx$ 150 bytes on the wire.
- The world's routable road network is on the order of **300 million directed edges** (two directions per road piece, worldwide).
- A raster map tile averages 20 KB; the world is mapped at zoom levels 0–20.

Derived numbers:

Quantity	Estimate	How
Map tile requests	$\approx$ 140k QPS avg, 700k peak	$200\text{M users} \times 60 \text{ tiles/day} \div 86,400 \text{ s}, \times 5 \text{ peak factor}$
Place search QPS	$\approx$ 5k avg, 25k peak	$200\text{M} \times 2/\text{day} \div 86,400, \times 5$
Route request QPS	$\approx$ 7k avg, 20k peak	$200\text{M} \times 3/\text{day} \div 86,400, \times 3$
GPS update ingress	$\approx$ 3M updates/s	$15\text{M navigators} \div 5 \text{ s per update}$
Ingress bandwidth	$\approx$ 450 MB/s	$3\text{M updates/s} \times 150 \text{ B}$
Road graph size	$\approx$ 20 GB	$300\text{M edges} \times 64 \text{ B per adjacency record}$
Naive tile storage	$\approx$ 29 PB	$\sum_{z=0}^{20} 4^z \approx 1.47 \text{ trillion tiles} \times 20 \text{ KB}$

KEY INSIGHT — What the math tells us

Three conclusions drive everything that follows. First, tile traffic is enormous but **the content barely changes** — a perfect caching workload; Section 2.6 exists to make 95%+ of those 700k QPS vanish into a CDN. Second, the road graph (20 GB) fits in the RAM of a handful of machines, so routing is an **in-memory** problem — but 20k route QPS $\times$ 3 CPU-seconds for a naive shortest-path search would need **60,000 CPU cores**, which no fleet absorbs; Section 2.8's whole purpose is to shave those 3 seconds down to 1 millisecond, at which point 20 cores suffice. Third, 3M GPS updates per second is far too much for any request/response design — it demands a streaming pipeline, which Section 2.12 builds.

2.6 Core Challenge I: Serving the Map

A user opens the app and sees their city. They drag the map; new streets slide in. They pinch; the view dives from the whole country to one neighborhood. Each of those moments requires the **right piece of the planet, rendered, in tens of milliseconds**. Rendering the world on the fly for every gesture of 200 million daily users is impossible; the entire solution rests on one idea: **precompute the map as tiny, independently addressable squares, and cache them everywhere**.

DEFINITION — Map tile

A small, fixed-size square of the map — conventionally 256 $\times$ 256 pixels (as an image) or the equivalent bundle of geometry (as data) — covering a specific rectangular patch of the Earth's surface. A screen full of map is assembled from a grid of tiles, like mosaic pieces. Tiles are the unit of storage, caching, and transfer for every mainstream map service.

DEFINITION — Zoom level

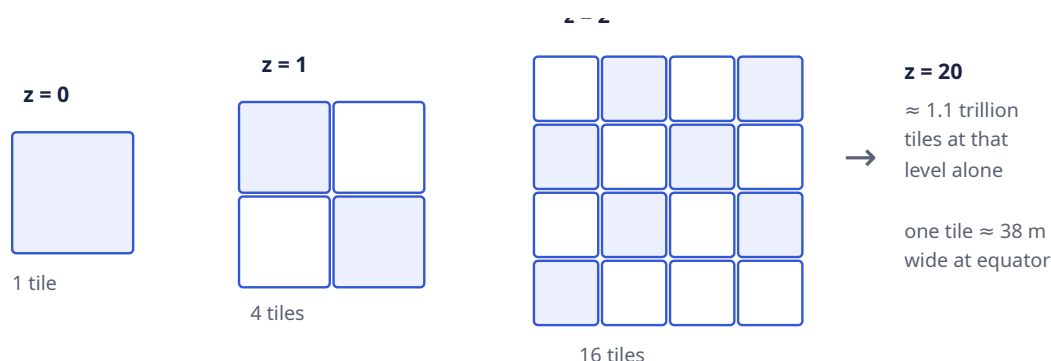
The map's scale, expressed as an integer z from 0 upward. At zoom 0 the **entire world** is one tile. Each step down splits every tile into four children, so zoom z has 2^z tiles per axis and 4^z tiles in total. Zoom 5

is roughly a country; zoom 10 a city; zoom 15 a neighborhood; zoom 20 shows a patch about 38 meters wide at the equator — individual buildings.

DEFINITION — Slippy-map tiling scheme

The near-universal convention for addressing tiles: every tile is identified by three integers (**z**, **x**, **y**) — its zoom level and its grid coordinates, counted from the top-left of the world. The Earth's surface is first flattened with the Web Mercator projection (which maps the sphere to a square and caps latitude near $\pm 85.05^\circ$), then cut into the $2^z \times 2^z$ grid. Given a latitude/longitude and a zoom, **anyone can compute which tile contains it** with a closed-form formula — Section 2.14 implements it. The addressing is deterministic, so the client never asks the server “which tiles do I need?”; it already knows.

The pyramid shape is the whole storage story. Four zooms, drawn:



Our estimate said storing **all** of that naively costs 29 PB. Two observations make it tractable:

1. **Most tiles are uniform.** Roughly 70% of the planet is ocean; enormous areas are desert, ice, or forest. A solid-blue 256×256 square compresses to almost nothing — and, crucially, **it is the same square everywhere**. Storing tiles content-addressed (Section 2.10) means every identical tile in the world is stored **once**; the trillions collapse to the few percent of tiles that actually contain roads and labels.
2. **Nobody looks at most of the world, most of the time.** Demand is violently skewed toward populated areas and common zooms. That skew is what caching feeds on.

DEFINITION — Content Delivery Network (CDN)

A geographically distributed fleet of **edge caches**: servers in hundreds of cities, each holding copies of popular content close to users. A request is routed to the nearest edge location; on a **cache hit** the content is served locally (single-digit milliseconds, zero load on our infrastructure); on a **miss** the edge fetches from our origin once and caches it for the next requester. CDNs turn a read-heavy, mostly-static workload into someone else's bandwidth.

DEFINITION — TTL (time to live) / cache hit ratio

A cached entry's *TTL* is how long an edge may serve it before revalidating with the origin — the dial between freshness and load. The *hit ratio* is the fraction of requests served from cache. For base map tiles we choose long TTLs (days; roads move slowly) and expect hit ratios above 95%; the **traffic overlay** tiles of Section 2.12 get TTLs of a minute or two, because stale traffic is worthless.

COMMON PITFALL — Serving tiles from your own fleet

The single most common junior mistake on this problem: drawing a “tile service” that renders or reads tiles on demand per request. At 700k peak QPS that fleet is vast, slow for distant users, and pointless — the content is static and cacheable. The correct shape is: **pre-render tiles offline, push them to object storage, put a CDN in front, and let the origin see a single-digit percentage of traffic**. Rendering is a batch job, not a request path.

One design fork is worth naming here and deciding later (Section 2.17): tiles can be **raster** (pre-rendered images) or **vector** (compact geometry the client renders on its GPU). We will serve vector tiles to modern clients — roughly half the bytes, restyleable without re-rendering, and one tile set serves every zoom-adjacent gesture smoothly — while keeping raster fallbacks for old clients. Either way, the tiling, addressing, and CDN story is identical.

Place search (FR-2) rides on the same geospatial ideas: a **spatial index** lets us find “all places inside this patch of Earth” without scanning 200M records. Section 2.7 defines the indexing structure once, because routing needs it too.

2.7 Core Challenge II: Modeling the Road Network

Directions require a mathematical object we can search. That object is a graph, built from the road network:

DEFINITION — Graph / weighted directed graph

A *graph* is a set of **nodes** connected by **edges**. It is *directed* when edges have a direction (a one-way street is traversable only along its arrow) and *weighted* when every edge carries a number measuring the **cost** of crossing it. We model intersections as nodes and stretches of road between them as edges, and we call one such directed road piece a **segment**.

DEFINITION — Segment

The atomic unit of the road network: one directed piece of road between two nodes, carrying metadata — geometry (the polyline of its shape), length, road class (residential, arterial, highway), speed limit, and turn restrictions at its far end. Segments are also the unit of **traffic**: when we say “this road is slow,” we mean “this segment’s current crossing time is high,” and every GPS update we receive is attributed to exactly one segment (Section 2.12).

The crucial choice is the edge **weight**. Distance is stable but wrong for drivers; what users minimize is **time**. So an edge’s weight is its **expected crossing time**: $\text{length} / \text{current expected speed}$. “Current expected speed” is where live traffic enters — the weight is $\text{length} / \text{speed_limit}$ on an empty road, and degrades as Section 2.12’s pipeline reports slower observed speeds. Routing and traffic are thereby one mechanism: **traffic is just edge weights that move**.

DEFINITION — Adjacency list

The standard memory layout for sparse graphs: for every node, a list of its outgoing edges. Routing graphs are extremely sparse (an intersection has 3–6 outgoing segments), so an adjacency list stores 300M small records — the 20 GB from Section 2.5 — instead of the quadratically-sized matrix a dense representation would need. Our routing engine holds adjacency lists **in RAM**; there is no database on the per-request path.

A subtlety interviewers enjoy: a graph edge cannot express “you may enter this intersection from Main St but not turn left onto 1st Ave.” Turn restrictions are handled either by splitting nodes (one

node per incoming direction, with only legal turns as edges) or by per-edge metadata the search consults. Either way, the model — nodes, directed edges, time weights — survives intact.

2.8 Core Challenge III: Shortest Paths at Planetary Scale

DEFINITION — Shortest path problem

Given a weighted graph and two nodes, find the path between them whose total edge weight is minimal. With time-weighted segments, the shortest path is the **fastest route**, and its total weight is the route's ETA. This is the single computation at the heart of FR-3 and FR-4.

The canonical algorithm, and the one the interviewer expects you to derive:

DEFINITION — Dijkstra's algorithm

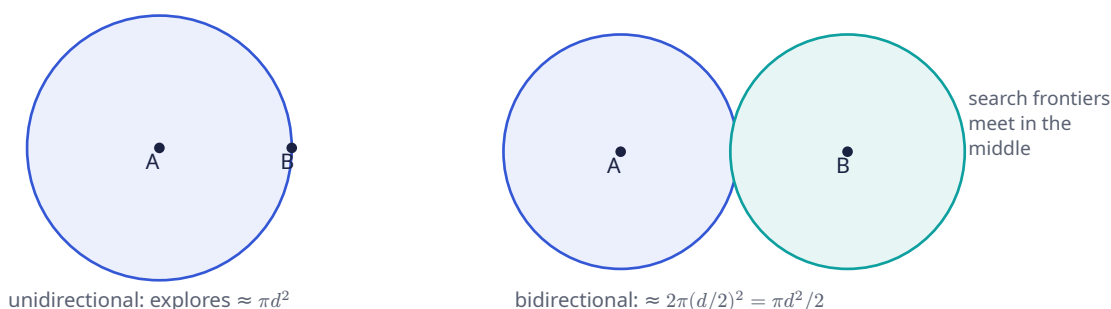
Explores the graph outward from the source in order of increasing distance from it. Maintain for each node the best known distance (initially ∞ , 0 for the source); repeatedly take the not-yet-finalized node with the smallest best-known distance, declare it final, and **relax** its edges — for each outgoing edge, check whether reaching its neighbor through this node beats the neighbor's best-known distance, and if so update it. With a min-priority queue selecting the next node, the running time is $O((V + E) \log V)$. It is provably exact for non-negative weights — and crossing times are always non-negative.

DEFINITION — Priority queue (min-heap)

A data structure supporting “insert with a priority” and “remove the minimum-priority element,” both in $O(\log n)$. A binary heap inside an array is the standard implementation. Dijkstra's algorithm spends its life asking “which unfinished node is currently closest?” — exactly this operation.

Dijkstra is correct — and unusable here. A cross-country query on 100M nodes takes seconds of CPU; Section 2.5 showed that times 20k QPS is 60,000 cores. Three refinements, each a layer of the same idea — **search less of the graph**:

Refinement 1 — bidirectional Dijkstra. Run two searches simultaneously: one forward from the origin, one **backward** from the destination over reversed edges. Stop when the frontiers meet; the route is the two half-paths joined. Why it helps is geometric:



A forward search from A must explore roughly every node within distance d of A — a ball whose node count grows like the **area** πd^2 . Two half-searches each explore a ball of radius $d/2$; together that is $2 \cdot \pi(d/2)^2 = \pi d^2/2$ — about **half** the work in a uniform graph, and better in road networks, where the frontiers approach each other along highways. Same exact answer, half the CPU, and trivially parallelizable across two threads.

Refinement 2 — A* search. Dijkstra explores equally in **all** directions — including straight away from the destination, which is obviously wasted. A* steers the search by reordering the priority queue.

DEFINITION — Heuristic / admissible heuristic

In A*, each node's queue priority is $\text{known distance from source} + h(\text{node})$, where the *heuristic* h estimates the remaining distance to the destination. The heuristic is *admissible* if it never **overestimates** the true remaining cost. Admissibility preserves Dijkstra's exactness guarantee while pulling the search frontier toward the goal — nodes in the right direction get popped first.

For a road network, the admissible heuristic is the **straight-line distance to the destination divided by the fastest speed anywhere in the network**: no legal route can beat the crow flying at top speed, so this h never overestimates. Computing it needs distances between points on a sphere:

DEFINITION — Haversine distance

The great-circle distance between two latitude/longitude points on a sphere — the length of the shortest path over the Earth's surface ("as the crow flies"). A closed-form trigonometric formula computes it in microseconds; Section 2.14 implements it. Haversine is our heuristic's yardstick and our fallback whenever "how far apart are these coordinates?" appears.

Refinement 3 — hierarchy: mega-segments, then contraction hierarchies. Even A* explores every side street near the origin and destination. But observe how **you** drive cross-country: local streets for a minute, then a highway for three hundred miles, then local streets again. The middle of a long route almost never touches small roads. Turn that observation into a mechanism: precompute and aggregate.

DEFINITION — Mega-segment

A precomputed synthetic edge that summarizes a chain of ordinary segments — for example, one edge meaning "highway, exit 12 to exit 47, 52 km, 31 minutes at current speeds." A long-distance search can then hop between on-ramps and off-ramps on mega-segments instead of expanding thousands of small segments. The source talk builds its long-range routing this way: segments roll up into mega-segments, which roll up further, so a query quickly climbs from street level to highway level, crosses the country on a handful of edges, and descends again. Mega-segment weights are sums of member segment weights — so when traffic updates a segment (Section 2.12), the change **bubbles up** to every mega-segment containing it.

The literature's formal, optimal version of the same idea:

DEFINITION — Contraction hierarchy (CH)

A preprocessed form of the road graph. Offline, order all nodes by importance (highway junctions outrank cul-de-sacs) and **contract** them in that order: removing a low-importance node, and — whenever the shortest path between two of its neighbors ran through it — inserting a **shortcut edge** with the summed weight, so all shortest-path distances are preserved. At query time, run bidirectional Dijkstra with one rule: only follow edges that go **up** in importance. The two upward searches meet at the most important node on the route. Preprocessing takes hours and adds 30–100% more edges; queries drop from seconds to **microseconds-to-milliseconds** — the roughly 1000× speedup Section 2.5's arithmetic demands. Production engines (OSRM, Valhalla, and the systems the source talk gestures at with mega-segments) all run variants of this playbook.

The full ladder, with the numbers that justify it:

Algorithm	Preprocessing	Query time	Verdict
Dijkstra	none	seconds (continental)	Teaches the principle; too slow to serve
Bidirectional Dijkstra	none	2× faster, still seconds	Free win; still too slow
A* + haversine	none	often 5–10× faster	Great for short trips; long trips still expand too much
Mega-segments / CH	hours, offline	≈ 0.1–1 ms	Production answer: the 1000× that makes 20k QPS fit on 20 cores

KEY INSIGHT — Precomputation is the theme of the whole chapter

Tiles are pre-rendered so reads are cache hits (Section 2.6); the graph is pre-contracted so queries touch only highways-in-the-middle (this section); traffic speeds are pre-aggregated per segment so ETAs are table lookups (Section 2.12). A maps system is a machine for moving work **off** the request path and into batch and stream pipelines. When an interviewer asks “how does it answer so fast?”, the honest one-word answer is: **earlier**.

2.9 API & Protocol Design

Chapter 1 split its API into a control plane and a data plane; the same split applies, with a twist: our heaviest “API” is not an API at all.

Tiles are plain GETs — and mostly not ours. The client computes the (z, x, y) address of every tile in view (Section 2.6) and issues ordinary `GET /v1/tiles/{z}/{x}/{y}` requests against a tile hostname. Nearly all of these terminate at a CDN edge and never reach us. This is why the tile endpoint has no interesting request parameters: all the intelligence is in the **addressing scheme**.

DEFINITION — Polyline encoding

A compact ASCII encoding of a sequence of coordinates — Google’s variant encodes latitude/longitude deltas as variable-length base-64-ish text, so a 500-point route is a few hundred bytes instead of a JSON array of floats. Routes are transmitted as encoded polylines and decoded client-side; the format exists because a route crosses a route-length’s worth of geography but must fit in one small response.

The remaining request/response endpoints (REST, as defined in Chapter 1):

Method	Path	Purpose
GET	<code>/v1/tiles/{z}/{x}/{y}</code>	Map tiles; served by CDN edge, origin on miss
GET	<code>/v1/search?q=&near=lat,lng</code>	Place search: text + spatial filter; ranked results
GET	<code>/v1/search/autocomplete?q=</code>	Prefix suggestions as the user types
POST	<code>/v1/routes</code>	Directions: origin, destination, mode, departure time → candidate routes with distance, ETA, ETA-in-traffic, steps, encoded polyline
POST	<code>/v1/geocode</code>	Address → coordinates (and reverse: coordinates → address)

A route request and its response:

```

POST /v1/routes
{
  "origin":      { "lat": 37.7749, "lng": -122.4194 },
  "destination": { "lat": 37.3861, "lng": -122.0839 },
  "mode": "driving",
  "depart_at": "now"
}

200 OK
{
  "routes": [{
    "route_id": "rt_9c31",
    "distance_m": 56300,
    "eta_seconds": 2820,
    "eta_in_traffic_seconds": 3450,
    "polyline": "a~l~Fjk~u0wIb@_...",
    "steps": [
      { "instruction": "Head north on Market St",
        "segment_ids": ["seg_7712", "seg_7713"],
        "distance_m": 400, "eta_seconds": 61 }
    ]
  }]
}

```

Navigation (FR-5) is a session, not a request: the phone streams its position up and the server streams guidance down. That is a bidirectional, long-lived, low-latency channel — the WebSocket data plane from Chapter 1, reused verbatim.

Message	Direction	Payload & semantics
nav_start	client → server	{route_id, auth} — begin the session on the chosen route
location_update	client → server	{lat, lng, speed_mps, heading_deg, t} — one GPS fix, every 5 s; drives guidance and the traffic pipeline
instruction	server → client	{text, voice, distance_to_maneuver_m} — the next turn, delivered early enough to speak
eta_update	server → client	{eta_seconds, remaining_m} — refreshed as segment weights move
reroute	server → client	{reason, new_route} — deviation or a newly faster alternative
nav_end	either	Arrived, cancelled, or the connection dropped

Two protocol notes. First, `location_update` is **one stream feeding two masters**: live guidance for this user, and (anonymized, aggregated) the traffic pipeline for everyone else — Section 2.12 draws that fork. Second, navigation is the one place we push **down** to a moving phone, which raises the question “which gateway holds this user’s connection?” — answered by the connection manager in Section 2.13.

2.10 Data Model & Storage

DEFINITION — Polyglot persistence

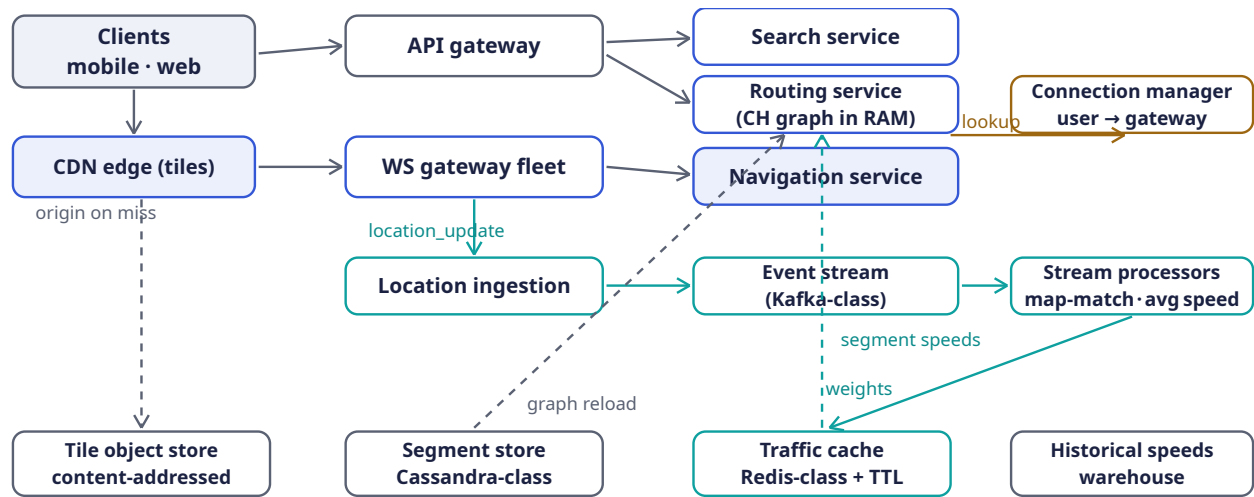
Using different storage engines for different access patterns within one system, instead of forcing every entity into one database. The cost is operational complexity; the benefit is that each workload gets the structure it actually needs. A maps system is the canonical case — our six entities below want six different shapes.

Entity	Contents	Store
Tile	$(z, x, y) \rightarrow$ rendered image or vector geometry	Object store behind the CDN; content-addressed so identical tiles are stored once
Place	name, category, address, location, ratings, hours	Text-inverted index $\times$ spatial index; 200M records
Segment	geometry, length, road class, speed limit, turn restrictions	Wide-column store (Cassandra-class), sharded by geography; read-heavy, batch-written
Road graph	adjacency lists + current weights + short-cuts	In RAM on routing nodes; rebuilt/swapped from the segment store + traffic cache
Traffic	segment_id $\rightarrow$ current average speed, sample count, updated-at	In-memory key-value cache (Redis-class) with a TTL — absence means “no fresh data,” triggering the historical fallback
Historical speeds	segment_id $\times$ (day-of-week, time bucket) $\rightarrow$ average speed	Analytics warehouse, batch-computed nightly from the GPS archive
Nav session	route, progress along it, last fix, ETA	In-memory on the navigation node; ephemeral like Chapter 1’s presence data

The ideas interviewers reward here: tiles are **content-addressed** (the 29 PB problem of Section 2.5 collapses because duplicate oceans are stored once); the routing graph lives **in memory**, with stores acting only as its source of truth during reloads; and the traffic cache’s TTL **is** the freshness guarantee — a segment whose entry has expired simply stops claiming live data.

2.11 High-Level Architecture

Section 2.4 promised separate planes; here they are. The **read plane** (tiles, search, routes) is stateless and cache-fronted. The **streaming plane** (GPS in, traffic out) is a pipeline. Navigation sits astride both.



Read plane: stateless, cache-fronted. Streaming plane: GPS in → traffic out. Dashed: control/reload paths.

Component responsibilities, and the reason each exists:

Component	Responsibility	Why it is shaped this way
CDN edge	Serve 95%+ of tile requests	Tiles are static and skewed — the textbook cache workload (Section 2.6)
API gateway	Auth, rate limiting, routing for REST endpoints	Stateless; also the paid-API front door, where per-customer quotas are enforced
Search service	Text × spatial place lookup	Read-heavy over a slowly-changing index; replicas scale it
Routing service	Compute routes + ETAs on the CH graph	Holds the graph in RAM ; scales by region shards and replicas (Section 2.15)
WS gateway fleet	Hold millions of navigation connections	Stateless connection holders, exactly as in Chapter 1
Navigation service	Per-session guidance: progress, instructions, ETA refresh, reroutes	Stateful per session but sessions are small and independent
Connection manager	Registry: user → which gateway	So the navigation service can push reroute to the right socket (Section 2.13)
Location ingestion	Absorb 3M updates/s, validate, anonymize	A buffer that decouples phone bursts from processing (Section 2.12)
Event stream	Durable, replayable pipe of GPS updates	Chapter 2 defines it below; lets many consumers read the same firehose

Component	Responsibility	Why it is shaped this way
Stream processors	Map-match pings to segments; average speeds; write traffic cache; bubble weights up mega-segments	The batch-quality work of traffic, done continuously

2.12 Deep Dive: The Live Traffic Loop

This is the subsystem that turns “a map” into “**current** conditions,” and it is the heart of the source talk. Follow one GPS update until it changes someone’s ETA.

DEFINITION — Event streaming platform (Kafka-class)

A distributed, durable, append-only log organized into **topics**; producers append events, and any number of independent **consumers** read them at their own pace. Topics are **partitioned** (we partition by geographic key, e.g. geohash prefix) so hundreds of consumer instances process the stream in parallel, each owning a disjoint slice of the world. It absorbs our 3M updates/second, decouples ingestion from processing, and — because it is replayable — lets us rebuild derived state after failures.

DEFINITION — Stream processing

Computing over data **as it arrives**, event by event or in small windows, rather than in scheduled batch runs over stored data. Here: every GPS update is map-matched and folded into a per-segment running average within seconds of leaving the phone.

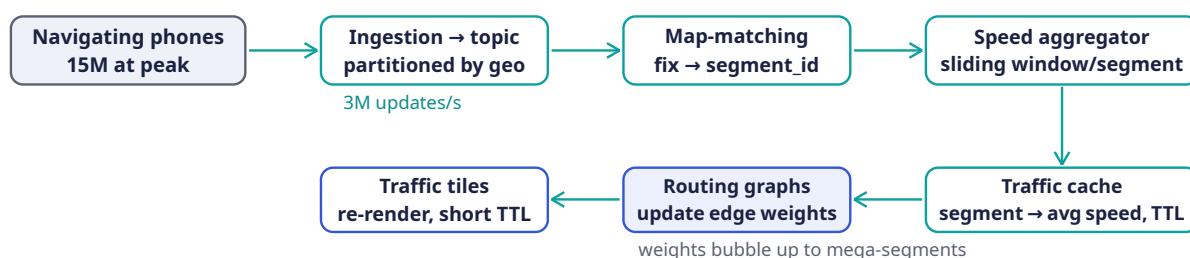
DEFINITION — Map-matching

Snapping a noisy GPS fix to the road segment the vehicle is most likely actually on. Raw GPS is wrong by 5–50 meters — enough to place a car on the wrong street or inside a building. The production-standard approach models the trace as a **hidden Markov model** (hidden states = candidate segments near each fix; emission scores = closeness of fix to segment; transition scores = plausibility of the implied movement) and solves it with the Viterbi algorithm. Map-matching is why the traffic pipeline’s first stage is a **processor**, not a database write.

DEFINITION — Sliding window

A moving time interval over a stream — “the last 15 minutes,” recomputed as time advances. A per-segment sliding-window average of observed speeds is our definition of “current speed”: old enough data falls out of the window and stops influencing the present. Section 2.14 implements the window.

The loop, end to end:



Every few minutes, reality → graph. ETAs and reroutes on new routes then use fresh weights automatically.

Three design details carry the interview:

1. **Speed, not position, is the aggregate.** Per segment and window, we average the speeds of map-matched vehicles. One slow vehicle is noise (a delivery van at the curb); the **average** over dozens of vehicles in 15 minutes is signal. This is also why privacy and accuracy align: nobody's individual trace is stored in the traffic cache — only segment statistics.
2. **Sparse segments fall back to history.** At 3 a.m. on a rural road, the window holds too few samples. Below a minimum count we blend toward the **historical** speed for that segment, day-of-week, and time-of-day — Tuesday-8 a.m. traffic is remarkably similar to last Tuesday-8 a.m. Live data where it exists, patterns where it does not, speed limits as the floor of last resort.
3. **Third-party feeds are scored, not trusted.** We also ingest incident and flow feeds from external providers. The talk's discipline: validate each feed against our own observed reality — if a provider claims congestion on a corridor where our measured speeds never dropped, that claim is wrong, and a provider that is wrong often gets **down-weighted or ignored** at the organization level. Every external signal is guilty until statistically innocent.

The same machinery paints the traffic overlay (FR-6): segment speeds become green/yellow/red classes, rendered into a **separate tile layer** with a TTL of a minute or two — short-lived tiles on top of the long-lived base map, each layer cached at its own freshness.

2.13 Deep Dive: The Turn-by-Turn Navigation Session

A navigation session is a small state machine per user, held in the navigation service: `ON_ROUTE` → `OFF_ROUTE_SUSPECTED` → `REROUTING` → `ON_ROUTE` → `ARRIVED`. Each `location_update` advances it:

1. **Project.** Map-match the fix onto the planned route's polyline: how far along the route is the user, and how far **off** it?
2. **Guide.** From progress along the route, emit the next `instruction` early enough to be spoken ("in 300 meters, turn right").
3. **Refresh.** Recompute the ETA over the remaining segments using **current** weights from the traffic cache; push `eta_update` when it moves by more than a small threshold (users notice a jumping ETA; hysteresis is a feature).
4. **Reroute.** If the fix sits beyond a distance threshold from the polyline **and** heading disagrees with the route, **sustained** for a few consecutive updates (one bad fix must not trigger a reroute — GPS noise is constant), recompute from the current position and push `reroute` with the new route. Section 2.14 implements this decision.

The push path needs one piece of plumbing: the navigation service knows **what** to send but not **where the user's socket lives**. The **connection manager** — a replicated key-value registry mapping `user_id` → `gateway_id` — answers that: gateways register each connection on connect and remove it on drop; the navigation service looks the user up and hands the message to that gateway. If the registry entry is stale (a gateway died), the client's reconnect registers a fresh one within seconds, and at worst one push is retried. This is Chapter 1's stateless-gateway story, completed with its directory.

COMMON PITFALL — Rerouting on a single GPS fix

GPS in a downtown canyon can teleport a user two blocks sideways for one update. A reroute fired on that ghost fix is a terrible user experience — and it also **pollutes the traffic pipeline** with a phantom slow-down on a road the user never touched. Both loops therefore consume **map-matched, sustained** signals, never raw single fixes. The same discipline, twice.

INTERVIEW TIP — Navigation availability is a degradation story, not an uptime story

Four nines on the navigation path does not mean “the server never dies”; it means **the phone copes when it does**. The client caches its route, its upcoming tiles, and its step list; if the session drops, guidance continues locally from the last known state while a new session is established. Saying “the client is the final fallback layer of the server” is exactly the kind of answer the 99.99% requirement is fishing for.

2.14 Deep Dive: Rust Reference Implementations

Four pieces of this chapter are small enough — and interview-critical enough — to show in full: tile addressing, the routing core, the traffic window, and the reroute decision.

2.14.1 Tile addressing: lat/lng → tile → quadkey

The slippy-map formulas of Section 2.6, plus the quadkey encoding that makes tiles prefix-searchable:

```
/// Web-Mercator slippy-map addressing (Section 2.6).

/// Which tile (x, y) at `zoom` contains this lat/lng?
pub fn lat_lng_to_tile(lat: f64, lng: f64, zoom: u8) -> (u32, u32) {
    let n = 2f64.powi(zoom as i32); // tiles per axis: 2^z
    let x = ((lng + 180.0) / 360.0 * n) as u32;
    let lat = lat.to_radians();
    let y = ((1.0 - lat.tan().asinh() / std::f64::consts::PI) / 2.0 * n) as u32;
    let max = n as u32 - 1;
    (x.min(max), y.min(max)) // clamp the antimeridian edge
}

/// The same address as a quadkey: one digit per zoom level, '0'..'3',
/// interleaving y/x bits from the most significant down. Neighboring tiles
/// share prefixes – a quadkey IS a quadtree path – so a plain sorted
/// key-value store can answer "all tiles in this area" as a range scan.
pub fn quadkey(x: u32, y: u32, zoom: u8) -> String {
    let mut key = String::with_capacity(zoom as usize);
    for level in (0..zoom).rev() {
        let mask = 1u32 << level;
        let mut digit = 0u8;
        if x & mask != 0 { digit += 1; }
        if y & mask != 0 { digit += 2; }
        key.push((b'0' + digit) as char);
    }
    key
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn whole_world_is_tile_zero() {
        assert_eq!(lat_lng_to_tile(0.0, 0.0, 0), (0, 0));
    }

    #[test]
    fn san_francisco_zoom_16() {
        // Verified against the slippy-map formulas.
        assert_eq!(lat_lng_to_tile(37.7749, -122.4194, 16), (10482, 25331));
        assert_eq!(quadkey(10482, 25331, 16), "023010203330032");
    }
}
```

```
#[test]
fn bing_canonical_quadkey() {
    // The documentation example: tile (3, 5) at level 3 -> "213".
    assert_eq!(quadkey(3, 5, 3), "213");
}
}
```

The quadkey is the bridge to place search: index every place under the quadkey of its location, and “places near me” becomes “places whose key shares my tile’s prefix,” widened one ring of neighbors at a time.

2.14.2 The routing core: haversine, Dijkstra, A*

Section 2.8’s algorithm ladder, executable. Weights are integer **milliseconds** of expected crossing time — exact, and exactly what the traffic loop updates.

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

pub type NodeId = u32;

/// One directed segment leaving a node (Section 2.7).
#[derive(Clone, Copy, Debug)]
pub struct Edge {
    pub to: NodeId,
    pub millis: u32, // expected crossing time – the weight traffic moves
}

/// In-memory adjacency-list road graph (Section 2.7).
pub struct RoadGraph {
    pub adj: Vec<Vec<Edge>>,
    pub lat: Vec<f64>, // node coordinates, for haversine and A*
    pub lng: Vec<f64>,
}

/// Great-circle distance in meters (Section 2.8).
pub fn haversine_m(a: (f64, f64), b: (f64, f64)) -> f64 {
    const R: f64 = 6_371_000.0; // Earth radius, meters
    let (la1, la2) = (a.0.to_radians(), b.0.to_radians());
    let dla = (b.1 - a.1).to_radians();
    let dlo = (b.1 + a.1).to_radians();
    let h = (dla / 2.0).sin().powi(2) + la1.cos() * la2.cos() * (dlo / 2.0).sin().powi(2);
    2.0 * R * h.sqrt().asin()
}

/// Dijkstra: exact shortest path by total crossing time.
pub fn dijkstra(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>>) {
    {
        let mut dist = vec![u64::MAX; g.adj.len()];
        let mut prev = vec![u32::MAX; g.adj.len()];
        let mut heap = BinaryHeap::new();
        dist[source as usize] = 0;
        heap.push(Reverse((0u64, source)));
        while let Some(Reverse((d, u))) = heap.pop() {
            if u == target { break; } // popped => finalized => optimal
            if d > dist[u as usize] { continue; } // stale heap entry
            for e in &g.adj[u as usize] {
                let nd = d + e.millis as u64; // relax the edge
                if nd < dist[e.to as usize] {
                    dist[e.to as usize] = nd;
                }
            }
        }
    }
}
```

```

        prev[e.to as usize] = u;
        heap.push(Reverse((nd, e.to)));
    }
}
}
if dist[target as usize] == u64::MAX { return None; }
let mut path = vec![target]; // walk predecessors back
while *path.last().unwrap() != source {
    path.push(prev[*path.last().unwrap() as usize]);
}
path.reverse();
Some((dist[target as usize], path))
}

/// A*: the same search, re-prioritized by an admissible heuristic —
/// remaining straight-line distance at the fastest plausible speed, so it
/// never overestimates (and, on a road graph, is *consistent*, which is what
/// licenses the early exit when the target is popped).
pub fn astar(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)> {
    const MAX_SPEED_MPS: f64 = 70.0; // ~250 km/h: faster than any legal road
    let t = target as usize;
    let h = |n: NodeId| -> u64 {
        let i = n as usize;
        (haversine_m((g.lat[i], g.lng[i]), (g.lat[t], g.lng[t]))) / MAX_SPEED_MPS * 1000.0
    } as u64;
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((h(source), 0u64, source))); // (priority, dist, node)
    while let Some(Reverse((_, d, u))) = heap.pop() {
        if u == target { break; }
        if d > dist[u as usize] { continue; }
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
                prev[e.to as usize] = u;
                heap.push(Reverse((nd + h(e.to), nd, e.to)));
            }
        }
    }
    if dist[t] == u64::MAX { return None; }
    let mut path = vec![target];
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[t], path))
}

```

And the test that demonstrates the chapter’s slogan — **traffic is just edge weights that move**:

```

#[cfg(test)]
mod tests {
    use super::*;

    /// A tiny city: two ways from node 0 to node 4, edges ~1.1-1.6 km.
    /// 0 -> 1 -> 2 -> 4 : 60 s + 60 s + 60 s = 180 s

```

```

/// 0 -> 3 -> 4      : 30 s + 120 s      = 150 s   (normally fastest)
/// (Times are consistent with the A* speed cap of 70 m/s, so the
/// haversine heuristic stays admissible on this graph.)
fn tiny_city() -> RoadGraph {
    let mut adj = vec![Vec::new(); 5];
    let e = |adj: &mut Vec<Vec<Edge>>, from: u32, to: u32, millis: u32| {
        adj[from as usize].push(Edge { to, millis });
        adj[to as usize].push(Edge { to: from, millis }); // two-way streets
    };
    e(&mut adj, 0, 1, 60_000);
    e(&mut adj, 1, 2, 60_000);
    e(&mut adj, 2, 4, 60_000);
    e(&mut adj, 0, 3, 30_000);
    e(&mut adj, 3, 4, 120_000);
    RoadGraph {
        adj,
        lat: vec![0.0, 0.0, 0.01, -0.01, 0.0], // rough grid around origin
        lng: vec![0.0, 0.01, 0.01, 0.01, 0.02],
    }
}

#[test]
fn dijkstra_and_astar_agree() {
    let g = tiny_city();
    assert_eq!(dijkstra(&g, 0, 4).unwrap().0, 150_000);
    let (millis, path) = astar(&g, 0, 4).unwrap();
    assert_eq!(millis, 150_000);
    assert_eq!(path, vec![0, 3, 4]);
}

#[test]
fn traffic_reroutes() {
    let mut g = tiny_city();
    // Congestion report: the 3 -> 4 segment now takes 600 s.
    g.adj[3].iter_mut().find(|e| e.to == 4).unwrap().millis = 600_000;
    g.adj[4].iter_mut().find(|e| e.to == 3).unwrap().millis = 600_000;
    // No code path changes – the "traffic-aware" router is the same
    // algorithm reading different weights.
    assert_eq!(dijkstra(&g, 0, 4).unwrap().1, vec![0, 1, 2, 4]);
}
}

```

2.14.3 The traffic window: from GPS fixes to edge weights

The aggregator at the heart of Section 2.12: a per-segment sliding window of observed speeds, the live/historical blend, and the weight the graph stores.

```

use std::collections::VecDeque;

/// Per-segment sliding-window speed average (Section 2.12).
pub struct SpeedWindow {
    samples: VecDeque<(u64, f64)>, // (timestamp_secs, observed speed m/s)
    window_secs: u64,              // e.g. 15 * 60
}

impl SpeedWindow {
    pub fn new(window_secs: u64) -> Self {
        Self { samples: VecDeque::new(), window_secs }
    }
}

```

```

pub fn add(&mut self, now: u64, speed_mps: f64) {
    self.samples.push_back((now, speed_mps));
    self.evict(now);
}

fn evict(&mut self, now: u64) {
    while let Some(&(t, _)) = self.samples.front() {
        if now.saturating_sub(t) > self.window_secs { self.samples.pop_front(); }
        else { break; }
    }
}

/// Current average, or None when there is too little live data –
/// the caller then blends in the historical pattern (Section 2.12).
pub fn average(&mut self, now: u64, min_samples: usize) -> Option<f64> {
    self.evict(now);
    if self.samples.len() < min_samples { return None; }
    let sum: f64 = self.samples.iter().map(|&(_, s)| s).sum();
    Some(sum / self.samples.len() as f64)
}

/// Effective speed for a segment: live average where we trust it,
/// the historical pattern for this day/time otherwise, and never above
/// the posted limit (we do not route people as if speeding were planned).
pub fn effective_speed(live: Option<f64>, historical: f64, limit: f64) -> f64 {
    live.unwrap_or(historical).min(limit)
}

/// The edge weight the routing graph stores (Section 2.7): expected
/// crossing time. Clamped away from zero so weights stay finite and
/// Dijkstra's non-negativity requirement is never endangered.
pub fn weight_millis(segment_length_m: f64, speed_mps: f64) -> u32 {
    (segment_length_m / speed_mps.max(0.5) * 1000.0) as u32
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn window_evicts_old_samples() {
        let mut w = SpeedWindow::new(900); // 15-minute window
        w.add(1_000, 30.0); // highway at full speed
        w.add(1_700, 8.0); // jam begins
        w.add(1_800, 9.0);
        // The t=1000 sample is now 800 s old: inside the window.
        assert_eq!(w.samples.len(), 3);
        w.add(2_000, 10.0);
        // t=1000 is 1000 s old: evicted. Average over the jam only.
        let avg = w.average(2_000, 3).unwrap();
        assert!((avg - 9.0).abs() < 0.01);
    }

    #[test]
    fn sparse_segments_fall_back_to_history() {
        let mut w = SpeedWindow::new(900);
        w.add(100, 22.0); // one car at 3 a.m.
        assert_eq!(w.average(200, 10), None); // not enough samples
        // Historical pattern says 20 m/s here at this day/time; limit 25.
    }
}

```

```

    assert_eq!(effective_speed(None, 20.0, 25.0), 20.0);
    // Live data says 30 m/s, but we never plan on speeding.
    assert_eq!(effective_speed(Some(30.0), 20.0, 25.0), 25.0);
}

#[test]
fn weight_is_crossing_time() {
    assert_eq!(weight_millis(1_000.0, 10.0), 100_000); // 1 km at 10 m/s
    assert!(weight_millis(1_000.0, 0.0) > 0);          // clamped, finite
}
}

```

2.14.4 The reroute decision

Section 2.13's state machine, reduced to its durable core: distance from the planned polyline, sustained across fixes. (Distance uses a local flat-earth approximation — within a few hundred meters of the route, curvature is noise; production systems run the full map-matching of Section 2.12 here.)

```

/// Approximate meters-per-degree at a given latitude: 111.32 km per degree
/// of latitude everywhere; longitude degrees shrink with cos(latitude).
fn meters_per_deg(lat: f64) -> (f64, f64) {
    (111_320.0, 111_320.0 * lat.to_radians().cos())
}

/// Distance from point p to segment a-b, in meters (local approximation).
fn point_segment_m(p: (f64, f64), a: (f64, f64), b: (f64, f64)) -> f64 {
    let (m_lat, m_lng) = meters_per_deg(p.0);
    let (px, py) = (p.1 * m_lng, p.0 * m_lat);
    let (ax, ay) = (a.1 * m_lng, a.0 * m_lat);
    let (bx, by) = (b.1 * m_lng, b.0 * m_lat);
    let (dx, dy) = (bx - ax, by - ay);
    let len2 = dx * dx + dy * dy;
    let t = if len2 == 0.0 { 0.0 } else {
        ((px - ax) * dx + (py - ay) * dy) / len2).clamp(0.0, 1.0)
    };
    let (cx, cy) = (ax + t * dx, ay + t * dy); // closest point on a-b
    ((px - cx).powi(2) + (py - cy).powi(2)).sqrt()
}

/// Distance from a fix to the nearest point anywhere on the route polyline.
pub fn distance_to_route_m(p: (f64, f64), route: &[(f64, f64)]) -> f64 {
    route.windows(2)
        .map(|w| point_segment_m(p, w[0], w[1]))
        .fold(f64::MAX, f64::min)
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Guidance { OnRoute, OffRouteSuspected, Reroute }

/// The hysteresis filter: ONE noisy fix must never trigger a reroute
/// (Section 2.13's pitfall). `strikes` counts consecutive off-route fixes.
pub struct RerouteFilter {
    pub distance_threshold_m: f64, // e.g. 40 m
    pub required_consecutive: u8,  // e.g. 3 fixes
    strikes: u8,
}

impl RerouteFilter {
    pub fn new(distance_threshold_m: f64, required_consecutive: u8) -> Self {
        Self { distance_threshold_m, required_consecutive, strikes: 0 }
    }
}

```

```

    }

    pub fn update(&mut self, fix: (f64, f64), route: &[(f64, f64)]) -> Guidance {
        if distance_to_route_m(fix, route) > self.distance_threshold_m {
            self.strikes += 1;
        } else {
            self.strikes = 0;
        }
        if self.strikes >= self.required_consecutive { Guidance::Reroute }
        else if self.strikes > 0 { Guidance::OffRouteSuspected }
        else { Guidance::OnRoute }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    // A straight eastbound route along lat 37.0.
    fn route() -> Vec<(f64, f64)> { vec![(37.0, -122.0), (37.0, -121.9)] }

    #[test]
    fn on_route_fix_stays_quiet() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0001, -121.95), &route()), Guidance::OnRoute);
    }

    #[test]
    fn sustained_deviation_reroutes() {
        let mut f = RerouteFilter::new(40.0, 3);
        // ~56 m north of the route: beyond threshold.
        let lost = (37.0005, -121.95);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::Reroute);
    }

    #[test]
    fn one_ghost_fix_recovers() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0005, -121.95), &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update((37.0, -121.95), &route()), Guidance::OnRoute);
    }
}

```

2.15 Scaling & Geographic Sharding

Chapter 1 defined sharding; here the shard key is **geography itself**.

Tier	Sharding strategy	Why
Tiles	No sharding logic at all — the (z,x,y) address space partitions naturally; CDN edges cache by URL	Addressing is the partition scheme; popularity skew does the rest
Road graph	Partitioned by region (continent → subregion); each shard is a full in-	A route is overwhelmingly inside one region; replicas add both capacity and failover

Tier	Sharding strategy	Why
	memory copy of its region's CH graph, replicated	
Cross-region routes	Solved hierarchically: mega-segments cross region borders at a handful of highway gateways	The hierarchy of Section 2.8 doubles as the distributed-systems answer
GPS stream	Topic partitioned by geohash prefix; each stream-processor instance owns a disjoint set of cells	Map-matching and aggregation are embarrassingly parallel over geography
Traffic cache	Sharded by <code>segment_id</code> hash; total size 500 MB	Tiny, hot, and latency-critical — keep it in memory, close to routing
Place index	Replicated per region; sharded by quadkey prefix within a region	Search is read-heavy and geographically scoped (Section 2.14)

KEY INSIGHT — Geography is a forgiving shard key

Unlike user data (Chapter 1's documents), geography does not move, grow legs, or go viral. A region's road graph changes on the scale of weeks; its traffic weights change on the scale of minutes but are **regenerated**, not migrated. The one genuinely hot spot — a stadium emptying at 11 p.m. — is absorbed inside one partition by the aggregation pipeline, not spread across the system.

2.16 Failure Modes & Recovery

Failure	Handling
WS gateway dies	Clients reconnect to any gateway; the connection manager re-registers them; the navigation service resumes pushes to the new socket. Unacked client updates are retried and deduplicated (Chapter 1's idempotency discipline).
Navigation node dies	Sessions are small (route + progress). The client's reconnect carries its last known position; a fresh node rebuilds the session from the route store and continues. User sees a pause, not a crash.
Stream backpressure / Kafka lag	Freshness degrades first and visibly : per-segment entries age past their TTL, and the system slides to historical speeds automatically — the fallback of Section 2.12 is the degradation mode, by design. Recovery replays the log.
Traffic cache loss	Rebuilt from the stream within one window (15 min); historical speeds serve in the meantime.
Routing node dies	Each region shard is replicated; traffic shifts to a replica. A lost node reloads its graph from the segment store plus the traffic cache (Section 2.10).
Tile origin down	The CDN keeps serving — 95%+ of requests never noticed the origin existed. <code>stale-while-revalidate</code> semantics make even expired tiles safe, since the base map changes weekly at worst.

Failure	Handling
Bad or spoofed GPS input	Per-segment sample thresholds ignore singletons; per-source scoring quarantines systematically wrong feeds (Section 2.12).
Full region outage	The phone is the last fallback: cached route, tiles, and step list keep guiding offline (Section 2.13) while sessions re-home to another region.

2.17 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Vector tiles over raster	50% fewer bytes; restyle without re-render; smooth zoom	Client GPU does the work; a raster fallback must exist for old clients
CH / mega-segments over plain A*	1000× query speedup; the only way 20k QPS fits a small fleet	Hours of preprocessing; shortcut weights must be rebuilt or bubbled-up when traffic moves — extra machinery (Section 2.8)
WebSocket push for navigation	Reroutes and ETA updates arrive instantly; one connection also carries GPS upstream	Millions of long-lived connections to hold and to register (connection manager)
Hybrid live + historical traffic	Coverage everywhere, graceful under sparse data and pipeline lag	Two data paths to keep consistent; blending logic to get right
Quadkey/geohash indexing	Proximity becomes a string-prefix range scan in ordinary KV stores	Cell-boundary artifacts — always query the neighboring too
Precompute over on-demand render	Tiles and hierarchy make reads O(cache hit)	A planet of batch jobs to run, version, and roll back

2.18 Observability & SLOs

DEFINITION — MAPE (mean absolute percentage error)

The average of $|\text{predicted} - \text{actual}| / \text{actual}$ over many predictions — the standard accuracy metric for forecasts, and the natural SLI for ETA quality. Per **completed** trip, we know both the ETA we gave and the realized travel time, so ETA MAPE is measurable exactly, per city, per hour, per route class. The source talk is emphatic on this point: you cannot improve what you do not score, and ETA accuracy is the metric this product lives by.

What we instrument, at minimum: **ETA error distribution** (predicted vs. realized, per region — the chapter's headline SLI, targeting the $\pm 10\%$ of Section 2.4); **route acceptance rate** (how often users actually drive the recommended route — a recommendation nobody takes is a signal something is wrong with the routes, per the talk's analytics discussion); reroute rate per session; tile cache hit ratio

and origin QPS; route latency p50/p99; pipeline lag (fix → weight update); traffic freshness (median age of live segment speeds); navigation session drops and recovery times.

2.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- “*Offline maps?*” — Ship regional bundles (tiles + contracted graph) to the device; routing runs locally; traffic and reroute quality silently degrade until reconnect. Everything we precomputed server-side is exactly what the bundle contains.
- “*Public transit?*” — A **time-dependent** graph (edges exist only when a vehicle does); Dijkstra generalizes, but production transit uses schedule-based algorithms (RAPTOR and friends). Name it, don’t build it.
- “*GPS noise in urban canyons?*” — The HMM map-matching of Section 2.12, plus sensor fusion (accelerometer, gyroscope, wheel ticks where available).
- “*Better ETAs?*” — Features into a learned model: current and historical speeds, weather, events, turn and signal penalties; graph neural networks model congestion **spreading** along the road graph. Google reported up to 50% ETA-error reductions in some cities from exactly this move — say the number as motivation, then defend the simple hybrid as the baseline it must beat.
- “*Walking and cycling?*” — Same graph, same algorithms, different edge eligibility and weights (stairs, bike lanes, no highways). The design’s generality is the answer.
- “*Location privacy?*” — Aggregate, don’t store: segment statistics instead of traces, retention limits on raw updates, minimum sample counts that double as k-anonymity for sparse segments.

If you remember five things:

1. The map is a precomputed pyramid of addressable squares; the CDN, not your fleet, serves it.
2. Roads are a time-weighted directed graph, and **traffic is just edge weights that move**.
3. Dijkstra is exact and unusable; bidirectional and A* are free wins; hierarchy (mega-segments / contraction hierarchies) is the 1000× that makes it a product.
4. Live traffic is a streaming pipeline: GPS fix → map-match → per-segment windowed average → weights and overlay tiles, with history as the fallback.
5. Navigation correctness lives on the phone: the client caches the route and survives the server.

2.20 Summary & Further Reading

We designed a maps and navigation service for 1B monthly users: a pre-rendered, CDN-served tile pyramid addressed by (z, x, y) and quadkeys; a time-weighted road graph held in memory and searched with an algorithm ladder that ends at contraction hierarchies; a live traffic loop that turns 3M GPS updates per second into per-segment speeds, edge weights, and overlay tiles, with historical patterns as the fallback; and a turn-by-turn navigation service that guides, refreshes ETAs, and reroutes with hysteresis — degrading to the phone when the network or we fail.

Primary source for this chapter:

- “**13: Google Maps**” — **Systems Design Interview Questions With Ex-Google SWE (Jordan has no life)** — the walkthrough this chapter expands.

Foundations worth reading:

- The OSRM and Valhalla open-source routing engines — contraction hierarchies in production code, not just papers.
- Geisberger et al., *Exact Routing in Large Road Networks Using Contraction Hierarchies* (2008) — the CH formulation.
- Newson & Krumm, *Hidden Markov Map Matching Through Noise and Sparseness* (2009) — the map-matching standard.

- Google’s S2 geometry library documentation, and Uber’s H3 — the real spatial indexes behind quadkey-style addressing.
- The Bing Maps tile system documentation — the canonical quadkey reference.
- Google DeepMind’s write-up on learning ETAs with graph neural networks (2020) — the ML direction for Section 2.19’s follow-up.

2.21 Chapter 2 Glossary

A one-glance index of every term this chapter defined. Chapter 1’s glossary is assumed; later chapters assume both.

Term	Meaning in one line
ETA	Predicted travel time / arrival time; the quality bar of this chapter
Freshness	Age of the data behind an answer; traffic targets minutes
Map tile	Fixed 256×256 square of map; the unit of storage, cache, transfer
Zoom level	Scale integer z ; each step splits every tile into four
Slippy-map scheme	Deterministic (z, x, y) tile addressing over Web Mercator
Quadkey	Tile address as a digit string; a quadtree path, prefix-searchable
CDN	Planet-wide edge caches that absorb static reads near the user
TTL / hit ratio	How long cache entries may live / fraction of requests they absorb
Content addressing	Identical tiles (oceans) stored once, by content hash
Polyline encoding	Compact ASCII encoding of a route’s coordinate sequence
Graph	Nodes + edges; directed and weighted for roads
Segment	One directed road piece between nodes; the unit of traffic
Adjacency list	Per-node outgoing-edge lists; the in-memory graph layout
Shortest path	Minimum-total-weight route; with time weights, the fastest path
Dijkstra’s algorithm	Exact breadth-by-distance search with a priority queue
Priority queue	$O(\log n)$ “give me the current minimum” structure
Bidirectional search	Two half-searches meeting mid-route; half the work
A* / heuristic	Dijkstra steered by a remaining-distance estimate
Admissible heuristic	Never overestimates; preserves exactness
Haversine distance	Great-circle distance between coordinates
Mega-segment	Precomputed synthetic edge summarizing a segment chain
Contraction hierarchy	Node-importance ordering + shortcuts; μ s–ms queries
Kafka-class stream	Durable partitioned event log; decouples and replays
Stream processing	Computing over data as it arrives, not in batch
Map-matching	Snapping noisy GPS fixes to the true road segment
Sliding window	Moving time interval defining “current” speed per segment
Polyglot persistence	Different engines for different access patterns

Term	Meaning in one line
Connection manager	Registry of which gateway holds which user's socket
MAPE	Mean absolute percentage error; the ETA accuracy SLI

Designing a Distributed Rate Limiter

PROBLEM SOURCE

This chapter solves the problem posed in the talk “[7: Design a Rate Limiter](#)” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs a rate-limiting service for a public API: why limiting is a business feature and not just a shield, the classic limiting algorithms, and how to enforce limits correctly across a fleet of stateless servers. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol details, and Rust reference implementations.

3.1 The Problem Statement

The interviewer looks up and says:

“Design a rate limiter. Our public API is used by a million developers, and we need to make sure no single caller can overwhelm the service — or use more than their plan allows.”

Chapters 1 and 2 designed user-facing products. This chapter designs a piece of **infrastructure** — a component that sits in front of every request and decides, in well under a millisecond, whether the request may proceed. The difficulty is not the concept (“count requests, reject past a threshold” is a five-line program on one machine). The difficulty is doing it **correctly and cheaply on the hot path of a distributed system**: fifty gateway servers must enforce one shared limit per caller, under concurrency, without meaningful latency, and without the limiter itself becoming the outage it exists to prevent.

DEFINITION — Rate limiting / throttling

Controlling the **rate** at which a caller may invoke a system: at most n requests per time window per identity (API key, user, IP address, tenant). Calls beyond the limit are rejected (or queued, or slowed — hence *throttling*). Rate limiting serves three distinct masters: **protection** (abuse, brute force, accidental stampede), **capacity** (one noisy caller must not starve the rest), and **monetization** (usage tiers are limits with a price tag). Keep all three in mind: they produce different requirements.

DEFINITION — Quota vs. rate limit

A *quota* is a budget over a long period (“100,000 calls per month”) used for billing and plan enforcement; it tolerates approximate, batched accounting. A *rate limit* governs short windows (“100 calls per second”) to shape live traffic; it must be enforced **now**, on the request’s critical path. This chapter is about rate limits; Section 3.18 discusses how quotas differ.

3.2 Scope & Clarifying Questions

The prompt spans everything from login-page brute-force protection to planetary DDoS absorption. Narrow it:

Speaker	Dialogue
Candidate	“What are we limiting — HTTP API calls, or something else like messages or bytes?”
Interviewer	“HTTP requests to our public REST API. Limit per API key.”
Candidate	“Are limits the same for everyone, or tiered — free keys vs. paid plans?”
Interviewer	“Tiered. Rules must be configurable per key and per route, and changeable without redeploying anything.”
Candidate	“Scale of the API being protected?”
Interviewer	“Peak 200,000 requests per second, spread across a fleet of about 50 stateless gateway servers. One million registered API keys.”
Candidate	“How exact must enforcement be? If a key is limited to 100 requests per second, is 101 a scandal?”
Interviewer	“Never reject a caller who is under their limit. Small overshoot during failures is acceptable; systematic overshoot is not.”
Candidate	“How much latency may the check add to each request?”
Interviewer	“A millisecond or two at most — this runs on every single request.”
Candidate	“And if the limiter’s own state store is down — block everything, or let traffic through?”
Interviewer	“Great question. Decide, and defend the decision.”

AGREED SCOPE

Per-API-key rate limits on a public REST API; tiered limits, configurable per key and per route, hot-reloadable; enforced **globally** across 50 stateless gateway nodes at **200k RPS** peak; $\leq 1\text{--}2$ ms added latency; **no false rejections**, bounded overshoot; and an explicit, defended policy for limiter failure (fail-open vs. fail-closed — Section 3.15).

INTERVIEW TIP — The last question is the interview

“What do we do when the limiter breaks?” separates candidates who build components from candidates who build **systems**. Ask it yourself if the interviewer doesn’t — every subsequent design decision (centralized store, local fallbacks, timeouts) is downstream of the answer.

3.3 Functional Requirements

Chapter 1 defined functional requirements; ours:

1. **FR-1 — Enforcement.** A caller exceeding its limit receives HTTP **429 Too Many Requests** with a `Retry-After` hint; everyone else passes through.
2. **FR-2 — Configurable rules.** Limits are defined per API key, optionally per route or method, with named tiers (free, pro, enterprise); an admin API changes them at runtime.

3. **FR-3 — Transparency.** Responses carry the caller's limit state (limit, remaining, reset time) in standard headers, so well-behaved clients can self-throttle **before** being rejected.
4. **FR-4 — Global enforcement.** The limit holds across the entire gateway fleet, not per server — a caller with 100 req/s cannot get 5,000 req/s by being load-balanced across 50 nodes.
5. **FR-5 — Burst policy.** Short bursts above the sustained rate are allowed up to a defined allowance (real traffic is bursty; rejecting every microburst punishes normal clients).
6. **FR-6 — No false rejections.** A caller under its limit is never rejected. (Overshoot is a soft failure; false rejection is a hard one — it breaks innocent customers deterministically.)

3.4 Non-Functional Requirements

Three qualities dominate, and they pull against each other — naming the tension is half the answer:

Quality	Target
Added latency	≤ 1 ms in-region, p99 — the check is on every request's critical path
Accuracy	Zero false rejections; steady-state overshoot within 1% of the limit; degraded-mode overshoot explicitly bounded
Availability of the API	The limiter must never be a new single point of failure: if the limiter fails, the API stays up (fail-open), with a local safety cap
Scale	200k RPS enforcement across 50 nodes; 1M keys; 100M+ active keys per day tolerable
Config freshness	Rule changes propagate to all gateways within seconds

DEFINITION — False rejection / overshoot

The two ways a limiter can be wrong. A *false rejection* denies a caller who is under the limit — an availability bug, visible and unforgivable. *Overshoot* allows more than the limit — a protection degradation, tolerable in small doses. Accuracy requirements should always be stated in this asymmetric vocabulary, because the two errors have opposite costs and different causes.

KEY INSIGHT — Latency, accuracy, availability: pick the trade-off consciously

Exact global counting wants a synchronous round trip to a strongly consistent store (latency + availability cost). **Zero added latency** wants purely local counting (accuracy cost: 50 nodes $\times$ local limit). **Maximum availability** wants fail-open everywhere (protection cost). Every real design is a point in this triangle; the rest of the chapter locates ours and prices it.

3.5 Back-of-the-Envelope Estimation

Assumptions (stated, per Chapter 1's discipline):

- Peak gateway traffic: **200k requests/sec**; every request needs exactly one limit check.
- 1M registered keys; up to **100M keys active in a day** (many keys are used once and idle for weeks).
- Typical limit: 100 req/s per key; token-bucket state per active key is two numbers plus the key itself — 50–60 bytes.
- A modern in-memory store serves 100k simple ops/sec per shard.

Derived numbers:

Quantity	Estimate	How
Check operations	200k ops/s peak	one per request; reads+writes combined
Store shards needed	4–6 (+ replicas)	200k ops/s ÷ 100k ops/s per shard, with headroom
State memory (counters)	≈ 6 GB worst case	100M active keys × 60 B
State memory (sliding log)	≈ 800 GB worst case	100M keys × up to 1,000 time-stamps × 8 B — infeasible
Added latency budget	≤ 1 ms	one in-region round trip, or zero with local state
Config size	a few MB	1M keys × rule record; trivially cacheable everywhere

KEY INSIGHT — The scarce resource is round trips, not hardware

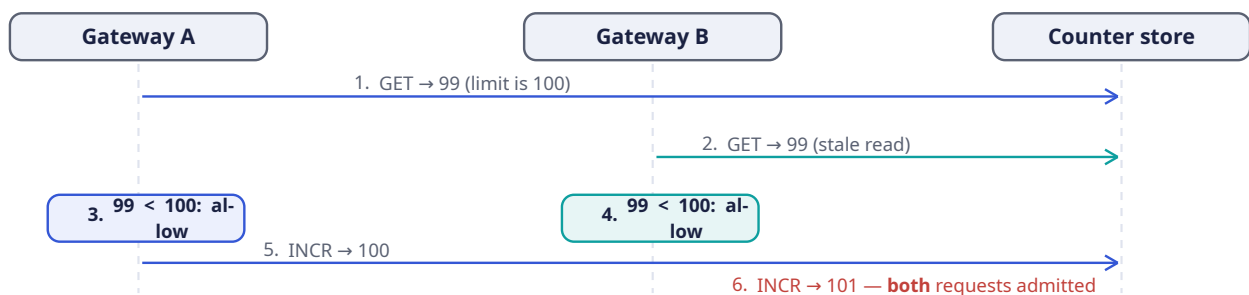
Six gigabytes of state and a few hundred thousand ops per second are, by Chapter 2's standards, nothing. The entire design problem is: **where does the check happen relative to the request?** A centralized exact check costs one network round trip (1 ms in-region — the whole budget). A local check costs zero round trips but is approximate. Estimation tells you this is a **latency topology** problem, not a capacity problem — design accordingly.

3.6 The Core Challenge: Counting Correctly Under Concurrency

The limit check is a read-modify-write: **read** the caller's count, **decide**, **write** the incremented count. Two naive placements fail in instructive ways.

Naive strategy 1 — count locally on each gateway. No shared state, zero added latency. But the limit is **per key**, and a key's requests land on all 50 gateways. If each node enforces 100 req/s locally, the caller effectively has **5,000 req/s** — FR-4 is unmet. Lowering each node's limit to $100/50 = 2$ req/s misfires the other way: a caller whose traffic happens to hit one node gets strangled. Per-instance limits cannot express a global limit.

Naive strategy 2 — a shared counter store with plain reads and writes. Put the counters in one shared, in-memory store; each gateway does **GET**, compare, **INCR**. This fails under concurrency:



DEFINITION — Race condition / check-then-act (TOCTOU)

A *race condition* is a bug whose outcome depends on the uncontrolled interleaving of concurrent operations. The specific species above is **time-of-check to time-of-use**: the decision (“count is 99, under the limit”) is based on state that another actor changes before our write lands. Both gateways checked

honestly; both were correct **at check time**; the limit was still exceeded. No amount of retries fixes a TOCTOU hole — the check and the update must become **one indivisible operation**.

DEFINITION — Atomicity / compare-and-swap (CAS)

An operation is *atomic* if it executes entirely or not at all, with no observable intermediate state. *Compare-and-swap* is the primitive form: “set this value to V_2 only if it currently equals V_1 ,” performed as one hardware/server-side step. Our options for making the check atomic: a server-side script executed atomically by the store (Section 3.12), a CAS loop, or a single atomic increment whose **returned value** is the decision input. The last two need no scripting at all — `INCR` already returns the post-increment count, which **is** the atomic observation we need.

So the core challenge decomposes into two questions that the rest of the chapter answers: **what exactly is the count** (which algorithm — Section 3.7), and **how is the check made atomic across the fleet** (Section 3.12).

3.7 The Algorithm Zoo

Five algorithms cover the entire design space. Each is a different answer to “what does ‘at most n per window’ mean, exactly?”

DEFINITION — Fixed window counter

Divide time into fixed windows (e.g., each clock minute) keyed `rl:{key}:{window_id}`; each request increments the current window’s counter; reject when it exceeds the limit. $O(1)$ memory per key, one atomic increment per request — and a famous flaw: **the boundary burst**. A caller can fire 100 requests at 0:59.9 and 100 more at 1:00.0 — 200 requests in 0.1 seconds, all legal, because the two bursts sit in different windows.

DEFINITION — Sliding window log

Store the **timestamp of every request** in a per-key log; on each request, drop timestamps older than the window and reject if the remaining count reaches the limit. Perfectly accurate, no boundary problem — but memory is $O(\text{limit})$ per key. Section 3.5 priced this at 800 GB at our scale: the log is the algorithm you describe to show you know the trade-off, not the one you ship.

DEFINITION — Sliding window counter

The fixed-window/log hybrid. Keep the current and previous window counters only, and estimate the sliding window as:

$$\text{estimate} = \text{current_count} + \text{previous_count} \times (1 - \text{elapsed} / \text{window})$$

If 15 seconds of a 60-second window have elapsed, the previous window’s 84 requests count as $84 \times 0.75 = 63$. $O(1)$ memory, one increment per request, no boundary burst — at the price of being an **estimate** (it assumes the previous window’s traffic was spread evenly, which is fair on aggregate; published analyses put the misclassification rate well under 1%).

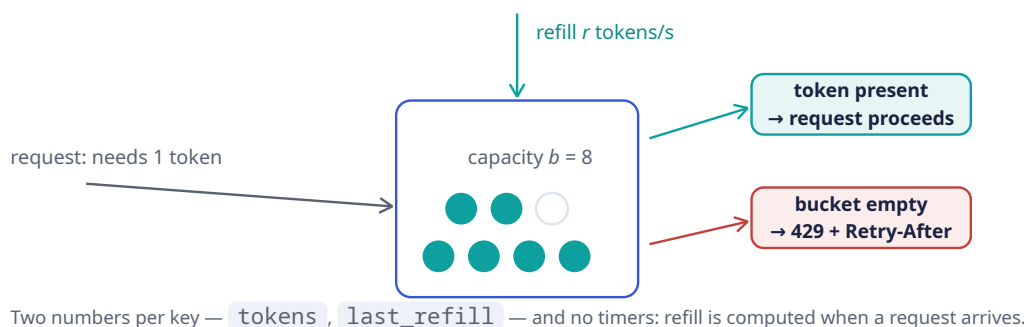
DEFINITION — Token bucket

Each key owns a bucket holding up to b tokens, refilled continuously at r tokens per second. A request takes one token; an empty bucket rejects. The parameters map directly onto product language: r is the sustained rate, b is the burst allowance (FR-5). $O(1)$ memory (two numbers: tokens, last-refill

timestamp), and refill is computed **lazily** on each request — no timers, no background jobs. This is the industry’s default for API limiting, and our choice; Section 3.11 deep-dives it and Section 3.13 implements it.

DEFINITION — Leaky bucket / GCRA

Requests enter a conceptual queue and “leak” out at a fixed rate; a request is admitted only if the queue has room. Where the token bucket **permits** bursts, the leaky bucket **forbids** them — output is perfectly smooth. Its stateless formulation is the **Generic Cell Rate Algorithm**: store one theoretical-arrival-time per key; admit if now is not earlier than that time minus tolerance. The right tool when you meter **into** a fragile downstream (payment gateways, SMS providers), overkill for request admission.



The comparison that ends the discussion:

Algorithm	Memory/key	Exact?	Bursts?	Notes
Fixed window	$O(1)$	window-granular	$2\times$ at boundary	Simplest; the boundary burst is the interview trap
Sliding log	$O(\text{limit})$	exact	no	Memory kills it at scale (Section 3.5)
Sliding counter	$O(1)$	1% error	smoothed	Best accuracy-per-byte when bursts are unwanted
Token bucket	$O(1)$	exact accounting	allowed up to b	Our choice: burst policy is a product feature here
Leaky / GCRA	$O(1)$	exact pacing	forbidden	Choose when downstream needs smooth flow

3.8 API & Protocol Design

A rate limiter’s “API” is mostly **other people’s responses**. The contract that matters:

DEFINITION — HTTP 429 / Retry-After

Status **429 Too Many Requests** (RFC 6585) means “you, specifically, are calling too fast.” The **Retry-After** header tells the caller how many seconds to wait before retrying. Together they convert

a rejection into a negotiation: a well-built client backs off exactly as instructed instead of hammering harder. A limiter that rejects without `Retry-After` trains clients to retry blindly — worse for everyone.

Header	Meaning
<code>X-RateLimit-Limit</code>	The caller's limit for this route, e.g. 100
<code>X-RateLimit-Remaining</code>	Requests left in the current window / tokens left
<code>X-RateLimit-Reset</code>	When the budget refills (epoch seconds or seconds-from-now)
<code>Retry-After</code>	On 429 only: minimum wait before retry, seconds

(An IETF draft standardizes these as `RateLimit-*` fields; the `X-` forms remain the de-facto convention. Either is defensible — knowing both exist is the point.)

A rejection response, end to end:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
Retry-After: 1
Content-Type: application/json

{ "error": "rate_limit_exceeded", "limit": 100, "window": "1s", "plan": "free" }
```

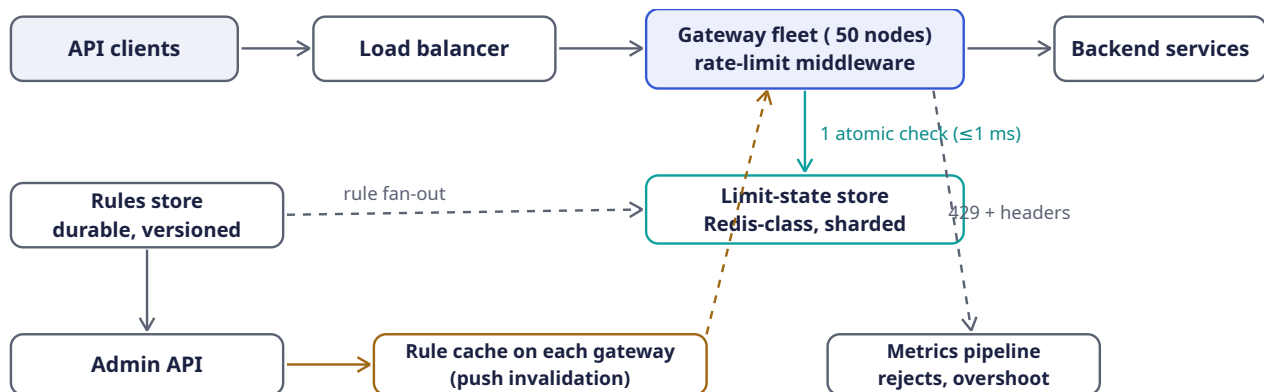
The **admin** surface (FR-2, FR-6) is ordinary REST: rules as records — `{ api_key or tier, route pattern, algorithm, limit/rate, burst, window }` — created and updated through an admin API, versioned, and pushed to every gateway within seconds (Section 3.10).

3.9 Data Model & Storage

Entity	Contents	Store
Limit state	<code>rl:{api_key}:{route}</code> → <code>{tokens, last_refill_ms}</code> (bucket) or window counters; TTL = window so idle keys self-clean	In-memory store cluster (Redis-class), sharded by key hash
Rule	tier or key → <code>{algorithm, rate, burst, window, route pattern}</code> ; versioned	Durable KV/relational store, fanned out to gateway-local caches
Quota ledger	monthly usage per key for billing	Append-only log → batch aggregation; not on the request path (Section 3.2)

Two decisions to name. First, state entries carry a **TTL** (Chapter 2) equal to the window, so the 100M-active-keys problem is self-cleaning: idle keys vanish without a garbage collector. Second, rules are cached **on each gateway** with push-based invalidation — a per-request rule lookup must never add a second network hop.

3.10 High-Level Architecture



The middleware is a read-modify-write on the hot path: one atomic store op per request, rules read locally.

Component	Responsibility	Why it is shaped this way
Rate-limit middleware	First code every request touches on the gateway: load rule → atomic check → pass or 429	Middleware placement means no backend code changes and one consistent policy for every route
Limit-state store	Atomic counters/buckets for all active keys	One shared store is what makes the limit global (FR-4); sharded by key hash, replicated for availability
Rules store + admin API	Versioned rule records; runtime updates	Config is tiny and read-hot: push it to gateway memory, never fetch per request
Gateway rule cache	Rules in process memory, invalidated by push	Zero added hops for config (Section 3.9)
Metrics pipeline	Every decision emitted as an event	Section 3.17: you cannot tune what you cannot see

KEY INSIGHT — The one-op rule

Per request, the design allows itself **exactly one** atomic store operation and zero synchronous config reads. Everything — the algorithm choice (O(1) state), the TTL self-cleaning, the gateway rule caches — exists to preserve that invariant. When an interviewer pushes on any component, return to it.

3.11 Deep Dive: The Token Bucket, Precisely

The token bucket's elegance is that **time is never advanced by a timer**. State is two numbers, updated lazily when a request arrives:

1. `tokens` — current balance, capped at capacity `b`.
2. `last_refill_ms` — when the balance was last recomputed.

On each request at time `now`:

```

elapsed_s = (now - last_refill_ms) / 1000
tokens    = min(b, tokens + elapsed_s * r) // lazy refill, capped
  
```

```

last_refill = now
if tokens >= 1: tokens -= 1; ALLOW (remaining = floor(tokens))
else:          DENY (retry_after_ms = (1 - tokens) / r * 1000)

```

Reading the parameters as product language: r = sustained rate (“100 req/s”), b = burst allowance (“... with bursts up to 150”). The retry hint is free: `retry_after` is exactly how long until one token exists.

A worked trace (limit 5 req/s, burst 8), to have in hand at the whiteboard:

Time	Event	Balance after	Why
<code>t = 0.0 s</code>	8 requests arrive together	$8 \rightarrow 0$	Full bucket: all 8 pass — the burst allowance
<code>t = 0.0 s</code>	9th request	0	Empty: 429, Retry-After: 1 (1/5 s until one token)
<code>t = 1.0 s</code>	1 request	$5 \rightarrow 4$	One second refilled 5 tokens
<code>t = 3.0 s</code>	1 request	$8 \rightarrow 7$	Two more seconds refill 10, capped at capacity 8

COMMON PITFALL — Refill by background timer

Implementing refill as a cron or timer per key means a million timers, clock drift between timer and request paths, and state that never cleans itself up. Lazy refill has none of these: no timers, no drift (one clock, read once per request), and a TTL on the key reclaims idle state for free. If you find yourself scheduling work per key, you have re-invented the sliding log’s costs inside the token bucket.

NOTE — One clock, and make it monotonic

Distributed machines disagree about wall-clock time (clock skew), and even on one machine the wall clock can jump backwards (NTP corrections) — a backward jump would **refund** tokens. Two defenses: (1) perform the bucket update **inside the store** with a server-side script, so one clock governs every gateway (Section 3.12); (2) where local time is unavoidable, read a **monotonic** clock (one that never moves backwards), as Section 3.13’s Rust does.

3.12 Deep Dive: Distributed Enforcement & the Race

Section 3.6’s race came from splitting **check** and **update** across a network. Three production-grade fixes, in increasing order of sophistication:

Fix 1 — atomic increment-as-decision. For window algorithms, one atomic `INCR` already returns the post-increment value: the store’s returned count **is** the check. Reject when the returned value exceeds the limit. One round trip, no client-side race, and the first increment of a window sets the key’s TTL in the same atomic step. Fixed window and sliding counter ship this way.

Fix 2 — server-side script for read-modify-write. Token bucket needs read-compute-write (refill, compare, decrement). Executing that sequence **on the store**, as one atomic script, removes the race identically: the store serializes script execution, so no two requests ever observe the same balance. Section 3.13 shows the client side, with the script embedded as data.

Fix 3 — approximate local counting. The radical option: skip the store on the hot path entirely. Each gateway keeps local counters and periodically **publishes** them; a background aggregator broadcasts fleet-wide totals, and each node denies when `local_estimate + fleet_total > limit`. Zero added latency, and the store is off the critical path (fail-open for free) — but between syncs, overshoot is

bounded by roughly $\text{limit} \times (\text{sync_lag} / \text{window})$. Choose it when protection matters more than precision (abuse mitigation); never for billing-adjacent limits.

Approach	Latency	Accuracy	Choose when
Centralized, atomic op	+1 RTT (1 ms)	exact	Default. Paid tiers, contractual limits
Centralized, non-atomic	+1 RTT	races under concurrency	Never — Section 3.6
Local + async sync	0	bounded overshoot	Abuse protection at extreme scale, or store-failure fallback
Sticky routing + local	0	near-exact	Only if your load balancer can pin keys to nodes — fragile on failover

INTERVIEW TIP — Say the quiet part: exactness is a product decision

“How exact does this need to be?” has no technical answer — it depends on whether the limit protects revenue (exact: paid tiers), infrastructure (approximate fine: abuse), or other users’ experience (approximate fine: fairness). Volunteering this framing turns an algorithm comparison into a senior-level requirements discussion.

3.13 Deep Dive: Rust Reference Implementations

Three pieces, all executable: the token bucket with deterministic-time tests, the fixed-window/sliding-counter pair showing the boundary burst and its cure, and the atomic store check with the server-side script embedded as data.

3.13.1 Token bucket with a testable clock

Time is injected through a `Clock` trait so tests control it exactly — the difference between a test that **suggests** correctness and one that **proves** it.

```
use std::sync::Mutex;

/// Time source, injected so tests can control it (Section 3.11).
pub trait Clock: Send + Sync {
    fn now_ms(&self) -> u64;
}

/// Production clock: monotonic — it can never jump backwards and refund
/// tokens (Section 3.11's note).
pub struct MonotonicClock(std::time::Instant);
impl MonotonicClock {
    pub fn new() -> Self { Self(std::time::Instant::now()) }
}
impl Clock for MonotonicClock {
    fn now_ms(&self) -> u64 { self.0.elapsed().as_millis() as u64 }
}

/// The decision every algorithm returns: pass, or reject with a retry hint
/// that becomes the Retry-After header (Section 3.8).
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Decision {
```

```

    Allow { remaining: u64 },
    Deny { retry_after_ms: u64 },
}

/// Token bucket (Section 3.7): `capacity` tokens, refilled lazily at
/// `refill_per_sec`. The Mutex makes the read-modify-write atomic *within*
/// one process; Section 3.12 handles the fleet-wide version.
pub struct TokenBucket<C: Clock> {
    capacity: f64,
    refill_per_sec: f64,
    state: Mutex<(f64, u64)>, // (tokens, last_refill_ms)
    clock: C,
}

impl<C: Clock> TokenBucket<C> {
    pub fn new(capacity: f64, refill_per_sec: f64, clock: C) -> Self {
        let now = clock.now_ms();
        Self { capacity, refill_per_sec,
            state: Mutex::new((capacity, now)), clock }
    }

    pub fn try_acquire(&self) -> Decision {
        let now = self.clock.now_ms();
        let mut s = self.state.lock().unwrap();
        let elapsed_s = (now - s.1) as f64 / 1000.0;
        let mut tokens = (s.0 + elapsed_s * self.refill_per_sec).min(self.capacity);
        s.1 = now;
        if tokens >= 1.0 {
            tokens -= 1.0;
            s.0 = tokens;
            Decision::Allow { remaining: tokens as u64 }
        } else {
            s.0 = tokens;
            let wait_ms = ((1.0 - tokens) / self.refill_per_sec * 1000.0).ceil() as u64;
            Decision::Deny { retry_after_ms: wait_ms.max(1) }
        }
    }
}

```

Tests, including the concurrency proof that `Mutex` buys us exactly-once accounting across threads:

```

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::{Arc, atomic::{AtomicU64, Ordering}};

    /// A clock the test controls explicitly.
    struct TestClock(AtomicU64);
    impl Clock for TestClock {
        fn now_ms(&self) -> u64 { self.0.load(Ordering::SeqCst) }
    }

    #[test]
    fn burst_then_reject_then_refill() {
        let clock = Arc::new(TestClock(AtomicU64::new(0)));
        let b = TokenBucket::new(8.0, 5.0, clock.clone()); // 8 burst, 5/s
        for i in 0..8 {
            assert!(matches!(b.try_acquire(), Decision::Allow { .. }), "burst #{i}");
        }
        // Bucket empty: denied, and told to wait 1/5s for one token.
    }
}

```

```

assert_eq!(b.try_acquire(), Decision::Deny { retry_after_ms: 200 });
// One second passes: 5 tokens return (capped at capacity).
clock.0.store(1000, Ordering::SeqCst);
assert_eq!(b.try_acquire(), Decision::Allow { remaining: 4 });
}

#[test]
fn exactly_capacity_across_threads() {
    let clock = Arc::new(TestClock(AtomicU64::new(0)));
    let b = Arc::new(TokenBucket::new(100.0, 1.0, clock));
    let allowed = Arc::new(AtomicU64::new(0));
    std::thread::scope(|s| {
        for _ in 0..8 {
            let (b, allowed) = (b.clone(), allowed.clone());
            s.spawn(move || {
                for _ in 0..25 { // 8 threads x 25 = 200 attempts
                    if matches!(b.try_acquire(), Decision::Allow { .. }) {
                        allowed.fetch_add(1, Ordering::SeqCst);
                    }
                }
            });
        }
    });
    assert_eq!(allowed.load(Ordering::SeqCst), 100); // never one more
}

```

3.13.2 Fixed window, its boundary burst, and the sliding counter's cure

```

/// Fixed window counter (Section 3.7): one counter per window per key.
pub struct FixedWindow {
    window_ms: u64,
    count: u64,
    window_start_ms: u64,
}

impl FixedWindow {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, count: 0, window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms; // window start
        if w != self.window_start_ms { // new window
            self.window_start_ms = w;
            self.count = 0;
        }
        self.count += 1;
        self.count <= limit
    }
}

/// Sliding window counter: previous window, weighted by overlap (Section 3.7).
pub struct SlidingWindowCounter {
    window_ms: u64,
    prev_count: u64,
    curr_count: u64,
    curr_window_start_ms: u64,
}

```

```
impl SlidingWindowCounter {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, prev_count: 0, curr_count: 0, curr_window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;
        if w != self.curr_window_start_ms {
            self.prev_count = if w == self.curr_window_start_ms + self.window_ms {
                self.curr_count // adjacent window: carry over
            } else { 0 }; // gap longer than a window
            self.curr_count = 0;
            self.curr_window_start_ms = w;
        }
        let elapsed = (now_ms - w) as f64 / self.window_ms as f64;
        let estimate = self.curr_count as f64
            + self.prev_count as f64 * (1.0 - elapsed);
        if estimate + 1.0 <= limit as f64 {
            self.curr_count += 1;
            true
        } else {
            false
        }
    }
}
```

And the test that **demonstrates the flaw** — fixed window admits a 2× burst at the boundary, sliding counter rejects it:

```
#[cfg(test)]
mod window_tests {
    use super::*;

    #[test]
    fn fixed_window_admits_boundary_burst() {
        let mut fw = FixedWindow::new(60_000); // 100 req/min
        // 100 requests at t = 59.5 s .. 59.9 s: all pass (window 0).
        for i in 0..100 {
            assert!(fw.try_acquire(59_500 + i, 100));
        }
        // 100 MORE at t = 60.0 s .. 60.4 s: all pass (window 1).
        // 200 requests in under a second – legal. That is the bug.
        for i in 0..100 {
            assert!(fw.try_acquire(60_000 + i * 4, 100));
        }
    }

    #[test]
    fn sliding_counter_rejects_the_same_burst() {
        let mut sw = SlidingWindowCounter::new(60_000);
        for i in 0..100 {
            assert!(sw.try_acquire(59_500 + i, 100));
        }
        // Just past the boundary the estimate is ~100 x overlap: the
        // previous window still counts almost fully. Burst denied.
        assert!(sw.try_acquire(60_200, 100));
        assert!(sw.try_acquire(61_000, 100));
        // Half a window later, weight has decayed: traffic flows again.
```

```

    let mut ok = 0;
    for i in 0..60 {
        if sw.try_acquire(90_000 + i * 500, 100) { ok += 1; }
    }
    assert!(ok > 40, "expected roughly half the limit to be available");
}

```

3.13.3 The fleet-wide atomic check

Within one process, a `Mutex` makes read-modify-write atomic. Across fifty gateways, the atomicity must live **in the store**: the whole check ships as one server-side script that the store executes without interleaving. In production Rust this is a string constant handed to the client library — the script is data; the engineering is Rust:

```

/// Fleet-wide fixed-window check (Section 3.12, Fix 1). Executed atomically
/// by the store, so the interleaving of Section 3.6 is impossible.
/// KEYS[1] = "rl:{api_key}:{route}:{window_id}", ARGV[1] = window seconds.
const FIXED_WINDOW_LUA: &str = r#"
    local n = redis.call("INCR", KEYS[1])
    if n == 1 then redis.call("EXPIRE", KEYS[1], ARGV[1]) end
    return n
"#;

/// One gateway node's decision: exactly one round trip, and the *returned*
/// count is the check — no client-side read-modify-write at all.
pub async fn allowed(
    con: &mut redis::aio::MultiplexedConnection,
    api_key: &str,
    route: &str,
    limit: u64,
    window_secs: u64,
    now_secs: u64,
) -> redis::RedisResult<Decision> {
    let window_id = now_secs / window_secs;
    let key = format!("rl:{api_key}:{route}:{window_id}");
    let n: u64 = redis::Script::new(FIXED_WINDOW_LUA)
        .key(key)
        .arg(window_secs)
        .invoke_async(con)
        .await?;
    Ok(if n <= limit {
        Decision::Allow { remaining: limit - n }
    } else {
        let reset = (window_id + 1) * window_secs;
        Decision::Deny { retry_after_ms: (reset - now_secs) * 1000 }
    })
}

```

Note what the code makes structural: the key **embeds the window id**, so window rollover is just a new key (old windows expire by TTL — self-cleaning, Section 3.9); and the retry hint falls out of the window arithmetic for free.

3.13.4 Where the check sits

The middleware contract, sketched against a generic handler: rules come from the gateway-local cache (zero hops), the decision from the fleet store (one hop), and `Deny` maps to the 429 contract of Section 3.8.

```

/// Gateway middleware shape (Section 3.10). `handler` is the real API.
pub async fn handle(
    req: Request,
    rules: &RuleCache,           // gateway-local, push-refreshed
    store: &mut Store,           // fleet-wide limit state
) -> Response {
    let rule = rules.for_key_and_route(req.api_key(), req.route());
    match store.check(req.api_key(), req.route(), &rule).await {
        Ok(Decision::Allow { remaining }) => {
            let mut resp = handler(req).await;
            resp.headers_mut().insert("X-RateLimit-Limit", rule.limit.into());
            resp.headers_mut().insert("X-RateLimit-Remaining", remaining.into());
            resp
        }
        Ok(Decision::Deny { retry_after_ms }) => Response::too_many_requests(
            retry_after_ms,       // -> Retry-After + X-RateLimit-Reset
            &rule,
        ),
        Err(_) => fail_open(req).await, // Section 3.15: never take the API down
    }
}

```

3.14 Scaling & Sharding

Sharding (Chapter 1) is by API key — the natural key, since every limit check is scoped to exactly one:

- **Limit-state store:** keys hash-sharded across 4–6 shards with replicas (Section 3.5’s arithmetic). Each check touches exactly one key, so there is no cross-shard coordination on the hot path.
- **Gateways:** stateless; any request can be checked anywhere, because the state lives in the shared store.
- **Rules:** fully replicated to every gateway — config is megabytes, and the read pattern (every request) demands local memory.

DEFINITION — Hot key

A single key whose traffic concentrates on one shard hard enough to matter — here, one viral API key doing tens of thousands of checks per second. A single atomic INCR or script call is O(1) and microseconds; a hot key is far more likely to be **rejected** fast than to melt a shard. If a key ever outgrows one shard, split its counter across sub-shards (`r1:key#0..15`) and sum at check time — the same trick Chapter 2’s stream used per segment.

Multi-region note. A truly global limit across regions forces synchronous cross-region round trips on the hot path — hundreds of milliseconds, violating the latency NFR. Production answer: enforce per-region limits sized to divide the global one, and reconcile asynchronously (Chapter 1’s PACELC framing: consistency would cost latency, so availability and latency win, and the **bounded overshoot** vocabulary of Section 3.4 is how you state the price).

3.15 Failure Modes & Recovery

DEFINITION — Fail-open / fail-closed

What a dependent system does when its dependency fails. *Fail-open* serves traffic anyway (availability over enforcement); *fail-closed* refuses (enforcement over availability). The choice is per-route: abuse protection on a public API fails open with a local cap; a login endpoint defending against brute force

fails **closed-ish** — refusing logins during a store outage is cheaper than admitting a credential-stuffing wave.

Failure	Handling
Limit-state store shard down	Fail over to the replica. If all replicas are gone: fail open with a per-gateway local token bucket sized $\text{limit} / \text{fleet_size}$ — traffic flows, protection degrades to a bounded multiple of the limit, metrics scream (Section 3.17).
Store unreachable from some gateways (partition)	Those gateways degrade to local caps; the rest enforce exactly. Converges on heal, no manual action.
Clock skew between gateways	Structurally impossible to matter: bucket/window math runs on the store's clock via the server-side script (Section 3.12); local code uses monotonic time (Section 3.13).
Bad rule pushed	Rules are versioned; roll back to the previous version. Rate of change is small, so a human-in-the-loop push is fine.
Rule fan-out stalls	Gateways enforce last-known rules and alert; a stale rule for 60 s beats a missing check for 60 s.
Retry storm after a 429 wave	<code>Retry-After</code> spreads retries; add jitter client-side guidance in docs. A limiter that herds retries into the same second recreates the burst it just rejected.
Hot key	Sub-shard the counter and sum (Section 3.14). Rare in practice — hot keys are usually already over their limit.

3.16 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Token bucket	Bursts as a product feature; O(1) state; lazy refill needs no timers	Two-number accounting must be atomic fleet-wide (script in the store)
Sliding window counter	No boundary burst, still O(1)	1% estimate error; slightly more math at check time
Fixed window alone	One atomic INCR; trivially explainable	2× boundary burst (Section 3.13's test proves it)
Centralized exact check	FR-4 truly holds; no false rejections	+1 RTT on every request; store on the critical path

Decision	Benefit purchased	Cost accepted
Local + async sync	Zero added latency; fail-open for free	Bounded overshoot; unusable for billing-adjacent limits
Fail-open default	The API never dies because the limiter did	During store outages, protection is a bounded approximation

3.17 Observability & SLOs

SLIs and SLOs (Chapter 1) for a limiter are unusual in one way: the system **working** looks like rejections. Instrument accordingly:

- **Decision rates** per rule: allows vs. 429s, over time. A 429 spike after a rule change is a config bug until proven otherwise.
- **Overshoot audit**: sampled per-key realized rates vs. limits — the direct measurement of Section 3.4's accuracy NFR.
- **Check latency** p50/p99, split by path (local rules lookup vs. store call); SLO: the ≤ 1 ms budget of Section 3.4.
- **Store health**: shard ops/sec, script latency, replica lag — the one dependency on the hot path.
- **Degraded-mode counters**: how many gateways are currently failing open, and with what local cap. This number should be zero; when it is not, paging is legitimate.
- **Top rejected keys**: the abuse list and the sales list are the same list — chronically limited keys are upgrade candidates (limits as monetization, Section 3.1).

3.18 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *“Per-key AND per-IP AND per-route at once?”* — Composite rules: evaluate each applicable limit; deny if any denies. Order checks cheapest-first.
- *“Monthly quotas for billing?”* — Different system: append-only usage log, batch aggregation, delayed enforcement. Rate limits shape traffic; quotas shape invoices (Section 3.1).
- *“Limiter as a shared service vs. a library?”* — A library removes a hop but re-introduces per-language drift and per-instance state; a sidecar/service centralizes policy at the price of the hop. State which you picked and why.
- *“Exactly-distributed without a central store?”* — Gossiped counters or CRDTs (Chapter 1) give you **approximate** global counts with zero coordination; exactness without coordination is not a thing, and saying so is the answer.
- *“L3/L4 DDoS?”* — Out of scope by layer: SYN floods and volumetric attacks are absorbed at the CDN/edge (Chapter 2) with connection-level defenses. This design is L7, per-authenticated-caller.
- *“Adaptive limits under load?”* — Load-shedding's cousin: when the backend browns out, tighten limits dynamically by a global multiplier. Nice extension; say it, don't build it live.

If you remember five things:

1. A rate limiter is a read-modify-write on the hot path; the design is the art of making that operation atomic and ≤ 1 ms.
2. Local counters cannot express a global limit; a non-atomic shared counter races. Atomicity lives **in the store**.
3. Token bucket = sustained rate + burst allowance, with lazy refill and self-cleaning TTL state.
4. Accuracy is asymmetric: never false-reject, bound the overshoot — and know that exactness is a product decision, not a technical one.
5. The limiter must never be the outage: fail open with a local cap, and page on degraded mode.

3.19 Summary & Further Reading

We designed a distributed rate limiter for a 200k-RPS public API: a gateway middleware making exactly one atomic check per request against a sharded in-memory store; the algorithm zoo (fixed window, sliding log, sliding counter, token bucket, leaky/GCRA) with token bucket chosen for its burst semantics; the TOCTOU race and its three fixes (atomic increment-as-decision, server-side scripts, approximate local counting); the 429 + `Retry-After` contract that turns rejections into negotiations; runtime-reloadable tiered rules pushed to gateway memory; and a defended fail-open policy with bounded local caps.

Primary source for this chapter:

- “7: Design a Rate Limiter” — [Systems Design Interview Questions With Ex-Google SWE \(Jordan has no life\)](#) — the walkthrough this chapter expands.

Foundations worth reading:

- Stripe’s engineering blog on rate limiters — production limiter design at API-company scale.
- Figma’s *An alternative approach to rate limiting* — the leaky-bucket-in-Redis variant, honestly costed.
- The IETF draft *RateLimit header fields for HTTP* — the emerging standard response contract.
- Brandur Leach’s writing on GCRA and the Redis-cell module — the stateless leaky bucket, implemented.
- NGINX’s rate-limiting documentation — the leaky bucket hiding inside a commodity reverse proxy.

3.20 Chapter 3 Glossary

A one-glance index of every term this chapter defined. Chapters 1–2 are assumed; later chapters assume all three.

Term	Meaning in one line
Rate limiting / throttling	Capping requests per identity per window; protection, capacity, monetization
Quota	Long-period budget for billing; batched, approximate accounting
False rejection	Denying an under-limit caller — the unforgivable error
Overshoot	Admitting over-limit traffic — the bounded, tolerable error
Race condition / TOCTOU	Check and use separated in time; state changes in between
Atomicity / CAS	All-or-nothing operation / conditional swap as one step
Fixed window counter	One counter per window; $O(1)$, but $2\times$ boundary bursts
Sliding window log	Every timestamp kept; exact; memory $O(\text{limit})$ — infeasible at scale
Sliding window counter	Two windows weighted by overlap; $O(1)$, 1% error, no burst
Token bucket	Capacity b refilled at r/s ; bursts allowed; lazy refill
Leaky bucket / GCRA	Constant-rate drip; bursts forbidden; smooth output
HTTP 429 / Retry-After	“Too fast” status + how long to wait — rejection as negotiation
X-RateLimit-* headers	Limit, remaining, reset — client-side self-throttling data
API gateway middleware	First code every request meets; policy without backend changes
Server-side script (Lua)	Read-modify-write executed atomically inside the store

Term	Meaning in one line
Local + async sync	Zero-latency approximate enforcement with bounded overshoot
Monotonic clock	Time source that never moves backwards; no refunded tokens
Fail-open / fail-closed	Dependency down: serve anyway / refuse — chosen per route
Hot key	One key concentrating load on one shard; sub-shard and sum
Tiered limits	Different limits per plan; limits as pricing
Rule cache	Gateway-local copy of config; zero hops per request
Key TTL self-cleaning	Idle keys expire with their window; no garbage collector
Boundary burst	Two legal window-fulls fired at a window edge — 2× in seconds
Degraded-mode cap	Local emergency limit while the fleet store is down